

Aircraft maintenance scheduling under uncertain task processing time

Matías Villafranca, Felipe Delgado *, Mathias Klapp

Department of Transport and Logistics, Pontificia Universidad Católica de Chile, Vicuña Mackenna 4860, Macul, Casilla 306, Correo 22, Santiago, Chile

ARTICLE INFO

Keywords:

Aircraft maintenance
Heuristic algorithm
Two-stage stochastic programming
Task scheduling

ABSTRACT

Unexpected delays while executing aircraft maintenance tasks can result in costly operational disruptions for airlines, including expensive flight delays and overtime. In this study, we address uncertainty in maintenance task processing times by designing a daily and cost-effective aircraft maintenance schedule using two-stage stochastic programming. In the first stage, we determine which daily maintenance tasks to outsource and which to complete with in-house technicians. Additionally, we schedule each task, outsourced or in-house, for its respective aircraft, specifying the start time and assigned maintenance base. In-house tasks are further assigned to a technician's work sequence. In the second stage, the start time of each task and the departure time of each flight are adjusted based on a specific realization of task processing times. We aim to minimize the expected costs incurred for outsourced maintenance tasks, overtime, and flight delays. To solve our model, we design an *ad-hoc* Adaptive Iterated Local Search heuristic that explores first-stage solutions via an efficient evaluation of the second-stage cost. We also present a proof of concept by testing our approach in a set of computationally simulated instances. Our proposed methodology yields 74% and 34% average cost savings compared to a deterministic approach assuming expected task processing times and to a conservative solution planning ahead with maximum possible processing times for each task, respectively. Moreover, we obtain 14% average cost savings compared to a benchmark solution, which plans maintenance tasks with an optimized time buffer parameter between consecutive task assignments. Furthermore, we study the cost impact of varying structural parameters, such as task granularity, processing time variability, workload, and cost structure.

1. Introduction

Strict maintenance safety standards regulate commercial air travel. Therefore, it is the safest passenger transportation mode (IATA, 2018) with an accident rate of one per 820 thousand flights in 2022 (IATA, 2023). In this context, planning aircraft maintenance operations is a significant economic challenge. In 2022, the global expenditure on aircraft maintenance reached \$79 billion of dollars (Prentice et al., 2022) accounting for 10% of airline's operational costs (IATA, 2022b). Besides minimizing direct costs, an efficient aircraft maintenance schedule should reduce the risk of unexpected operational disruptions, such as flight delays. In the US, the annual cost of delays ranges between 20 and 40 billion dollars (Ball et al., 2010; FAA, 2020), and maintenance schedules may be responsible for 10% of departure time delays (Eurocontrol, 2023). The two objectives, *i.e.*, minimizing direct costs and flight delays, can sometimes conflict. One should plan schedules that maximize aircraft and technician time utilization to minimize direct

* Corresponding author.

E-mail addresses: mavillafranca@uc.cl (M. Villafranca), fdb@uc.cl (F. Delgado), maklapp@uc.cl (M. Klapp).

<https://doi.org/10.1016/j.tre.2025.104012>

Received 9 April 2024; Received in revised form 1 February 2025; Accepted 4 February 2025

Available online 17 February 2025

1366-5545/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

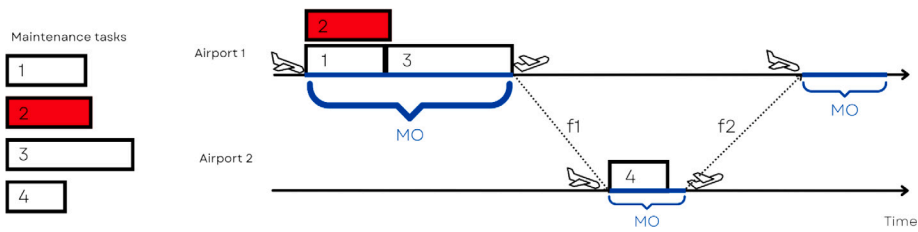


Fig. 1. Example of a line maintenance plan.

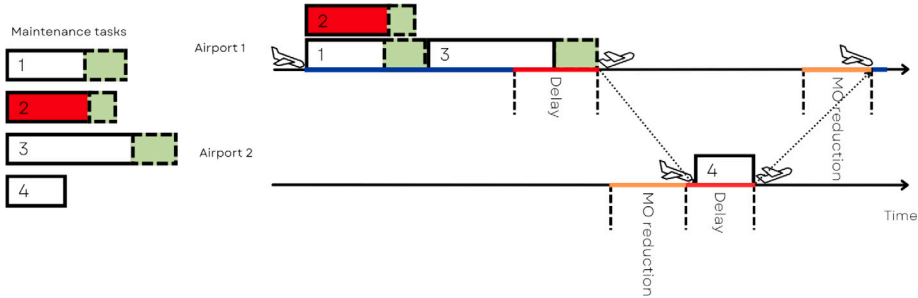


Fig. 2. Potential scenario for the line maintenance plan.

costs. Still, these plans may need time buffers to accommodate unexpected delays in executing maintenance tasks to avoid causing flight delays. Therefore, cost-efficient airlines should optimally balance both objectives.

Aircraft maintenance can be classified into periodic checks and line maintenance schedules. Periodic checks are routine task packages performed on each aircraft. Their duration varies from a few hours to months, depending on the check type (Qantas, 2016; Kinnison and Siddiqui, 2013). Checks are scheduled months in advance due to their significant requirements for hangar space and aircraft downtime. In contrast, line maintenance schedules perform individual and non-critical tasks during an aircraft's ground time window between two consecutive flights; it focuses on tasks that do not require hangar space. Line maintenance increases aircraft utilization as it reduces operational downtime. Historically, airlines relied primarily on periodic checks, but the focus has recently shifted towards line maintenance operations to improve efficiency (Ruther et al., 2017).

Typically, line maintenance plans are scheduled days in advance. Each task in the daily maintenance plan must be scheduled on an aircraft's available ground time window, which we call maintenance opportunity (MO). Accordingly, Papakostas et al. (2010), Lagos et al. (2020), Shaikat et al. (2020) have researched efficient line maintenance planning and developed variants of the Line Maintenance Scheduling Problem (LMSP) model. Typically, such a model selects which tasks to outsource and assigns a scheduled start time, maintenance base, and technician to each task. The LMSP is a complex scheduling problem. First, MOs are geographically spread in each aircraft itinerary, visiting multiple airports daily. Second, technicians are a limited resource geographically distributed over different maintenance bases, and each one can only execute a single task at a time without preemption. Third, multiple aircraft compete for technicians and bases.

Fig. 1 presents an example. It depicts the daily operation of a single aircraft visiting two airports on flights f_1 and f_2 . This flight itinerary offers three MOs to the maintenance. Suppose each airport has its own maintenance base and one available technician to execute maintenance tasks. For this aircraft, the maintenance planner must schedule tasks 1, 2, 3 and 4. Each task has an expected processing time represented by the length of each box on the left side of the figure. A feasible solution is to schedule tasks 1, 2 and 3 in the first MO and task 4 in the second MO. In this plan, task 2 is outsourced to a third-party maintenance provider, while the remaining tasks are performed in-house and assigned to the work sequence of the available technician at each airport. However, uncertain task processing times can disrupt the execution of a schedule, particularly when the plan is designed based on expected scenarios. If unexpected delays occur in tasks assigned to a specific MO, the realized processing time might exceed the MO's duration and potentially lead to flight delays. These delays could also propagate in time and space, affecting future aircraft MOs and flight departures. Fig. 2 depicts a propagated delay. The processing times of tasks 1, 2, 3 exceed their expected duration (by the area in green). Thus, the aircraft stays longer on the ground for the first MO. The resulting flight delay propagates to airport 2, reducing the available maintenance time in the subsequent MO and causing delays in task 4 start time and the next flight's departure.

To avoid flight delays, we may design a robust plan for the worst-case scenario. For example, one could assume that each task lasts its maximum possible processing time (derived from a known processing distribution). However, this strategy may probably lead to an expensive plan, translating into significant resource under-utilization.

Two-stage stochastic optimization is an alternative method that designs a maintenance schedule that minimizes the expected cost over multiple scenarios, each represented by a vector of task processing time realizations. This method can effectively reduce the number of outsourced tasks and proactively manage task time uncertainty, preventing flight delays. Despite the extensive use

of two-stage stochastic optimization in transportation planning problems, it has not been adapted to the LMSP under stochastic task processing times and multiple bases.

In this work, we address this gap by developing a methodology to plan efficient line maintenance schedules across multiple bases that proactively account for task processing time uncertainty. Our approach optimizes the balance between task outsourcing costs and the expected subsequent costs of overtime and flight delays. To do it, we re-formulate the LMSP as a two-stage stochastic programming model and propose exact and heuristic *ad-hoc* solutions for it. Our proposed methodology significantly outperforms a deterministic approach assuming expected task processing times and also a robust approach planning with the maximum possible processing times for each task. Moreover, it yields better results when compared to a typical heuristic method that allocates free time buffers between consecutively scheduled tasks to react to scenarios where tasks take longer than expected; this occurs, for any value of this buffer, including the optimal one constructed from offline training via simulation.

The remainder of this article is organized as follows. Section 2 offers a literature review focusing on aircraft maintenance. Section 3 presents our problem. Section 4 introduces our proposed model. In Section 5, the solution algorithms are described. Section 6 presents our case study, and Section 7 ends our paper with concluding remarks.

2. Literature review

We now present a literature review on aircraft maintenance planning, with a particular focus on line maintenance problems, and highlight the existing research gaps. Additionally, in Section 2.1, we discuss relevant methodologies to address these gaps, such as two-stage stochastic optimization and heuristics. Finally, we discuss our specific contributions in Section 2.2.

Most literature on aircraft maintenance centers on the Aircraft Maintenance Routing Problem (AMRP), a long-term preventive maintenance planning model to design each aircraft's itinerary and maintenance plan. Typically, decisions are taken months in advance, and, therefore, it aggregates maintenance (usually with periodic checks) without specifying each task's details, such as the start time. For related research on the AMRP, please refer to [Gopalan and Talluri \(1998\)](#), [Liang et al. \(2011\)](#), [Van den Bergh et al. \(2013\)](#), [Lagos et al. \(2020\)](#), [Saltzman and Stern \(2022\)](#). Some research articles on the AMRP have planned these long-term decisions, considering uncertainty in problem parameters. Unlike a deterministic problem, minimizing direct costs is insufficient. Instead, the focus extends to optimizing a given service level measured as expected flight delays and cancellations. In [Ma et al. \(2022\)](#), the authors study flight duration as the primary source of uncertainty. Multiple authors have designed robust strategies ([Lapp and Cohn, 2012](#); [Liang et al., 2015](#); [Ahmed et al., 2017](#)) and reactive strategies ([Başdere and Bilge, 2014](#); [Maher et al., 2014, 2018](#)) accordingly. However, uncertainty is not limited to flight duration. For instance, [He et al. \(2023\)](#) studies the combined uncertainty of task processing times and flight durations for the AMRP and proposes a robust approach.

Compared to the AMRP model intended for long-term decisions, our research focuses on an operational aircraft line maintenance planning problem. Despite the growth in airlines' demand to adopt a task-driven approach ([IATA, 2022a](#)), literature on line maintenance is limited. We classify the existing line maintenance literature into two categories. First, simulation tools that estimate a given maintenance plan cost and expected service level. These tools emulate real-world conditions and consider uncertain task processing times and their impact on potential flight delays. For instance, [Gupta et al. \(2003\)](#) tests different technician shift scheduling alternatives in a single airport and examines how the maintenance schedule responds to each option. [Papakostas et al. \(2010\)](#) presents a tool to evaluate a maintenance plan based on its direct cost and delay-induced costs. It models a setting with multiple geographically dispersed MOs and a fixed number of technicians available per airport. [Callewaert et al. \(2018\)](#) introduces an online simulation tool that dynamically sequences maintenance tasks based on revealed task processing times.

Second, optimization-related literature exists for the Line Maintenance Scheduling Problem (LMSP) model, intended to design cost-efficient line maintenance schedules. The LMSP has been studied isolated and integrated into other decision models, such as Workforce Scheduling (WSP), AMRP, and Tail Assignment Problem (TAP) models. [Safaei et al. \(2011\)](#) formulate an integer programming model intended for military applications. This model considers maintenance tasks disclosed before and after each aircraft flight. Each task must be executed to avoid keeping an aircraft on the ground. Compared to our research, it assumes deterministic task processing times and focuses on the single base problem. [Shahmoradi-Moghadam et al. \(2021\)](#) extend this work to uncertain task processing times. They employ a scenario-based strategy in which a deterministic solution is designed for each simulated scenario. Then, the cheapest feasible solution is selected over all scenarios. This study also centralizes all maintenance activities at one base. Consequently, it overlooks the geographic complexity of flight itineraries and delays. Regarding LMSP models for commercial airlines, [Shaukat et al. \(2020\)](#) formulate an MIP for the deterministic LMSP targeted for a 2 to 4-week maintenance planning operation. This study distinguishes between critical and non-critical tasks and aims to schedule each task as close to its due time as possible. This due time is set before the actual deadline of each task and is set by decision-makers as a security buffer. However, it omits task processing time uncertainty and assumes that flight departure and arrival times are fixed.

The WSP model is intended to plan a shift for each technician given a daily workload of tasks. A simultaneous solution of the WSP and LMSP models can be advantageous, given that a task schedule determines the daily workload. The work of [Beliën et al. \(2012\)](#) introduces a heuristic solution to a combined WSP-LMSP model. This work is extended to consider stochastic flight arrival times in [De Bruecker et al. \(2015\)](#) and to multiple technician skills in [De Bruecker et al. \(2018\)](#). However, these studies assign technician shifts to time windows rather than scheduling individual tasks. For instance, they presume an aggregated and fixed demanded workload for each MO. Moreover, they assume deterministic task processing times and a single maintenance base.

Also, the LMSP has been integrated with both AMRP and TAP models to reduce maintenance costs. [Safaei and Jardine \(2018\)](#) formulate a MIP model integrating these three problems. Due to the tactical nature of routing and TAP decisions, maintenance needs are aggregated in terms of the total person-hours required to execute a list of tasks that are about to expire. Therefore, the sequencing

and timetabling of individual tasks is not considered. Furthermore, unlike our research, it does not account for uncertainties in task processing times. Lagos et al. (2020) integrate the LMSP and TAP models where maintenance tasks are dynamically disclosed. Their study focuses on a unique maintenance base and assumes deterministic task processing times. So, potential flight delays due to maintenance are not considered.

While military aviation research encompasses LMSP models with task time uncertainty, it does not fully consider the specific challenges faced by commercial aviation, such as the cost involved in propagated delays and multiple and geographically dispersed bases with limited technician resources. In contrast, our work introduces a flexible LMSP model for commercial airlines that addresses uncertain task processing times, multiple bases, and task-to-technician assignments with scarce technician resources over time and geographically spread over multiple bases. We focus on scheduling individual tasks, considering that these tasks must be assigned to a technician's work schedule at a specific base. Additionally, we consider the propagated effect that an unusual delay in task processing times may have on subsequent task schedules and flight departures. Also, our approach does not overlook the network effect and the detailed scheduling complexity of integrated LMSP and WSP decisions. It differs from existing studies that aggregate the maintenance demand and the technician supply or that consider a single base.

2.1. Relevant methodologies

In our study, we employ a Monte Carlo sampling method (Shapiro, 2003) to generate possible scenarios of task processing times and feed a two-stage stochastic programming model. Specifically, we employ Sample Average Approximation (SAA) to approximate the expected second-stage cost using a sampled set of scenarios (Kleywegt et al., 2002; Pflug and Pichler, 2014). For more insights on stochastic optimization, consult Birge and Louveaux (2011), Shapiro et al. (2014).

We also complement our stochastic optimization framework with an Adaptive Iterated Local Search (AILS), a randomized local search heuristic. We explore a solution's neighborhood by performing local search moves. This way, we build multiple candidate solutions and choose the best among them. The efficacy of Local Search has been validated in challenging combinatorial problems, such as job shop scheduling, vehicle routing, graph coloring, and other combinatorial problems (Vaessens et al., 1996; Pisinger and Ropke, 2007; Caramia and Dell'Olmo, 2008; Dayarian et al., 2016; Gu et al., 2022). For a deeper understanding of heuristics and Local Search, refer to Gendreau et al. (2010).

2.2. Contributions

Our specific contributions are listed as follows: (i) We consider uncertain task processing times and assess their expected impact on flight delays when planning cost-efficient aircraft maintenance schedules over multiple bases and limited technician resources per base. To do it, we formulate the LMSP as a two-stage stochastic programming model. In the first stage, we determine which daily maintenance tasks to outsource (at a penalty) and which to complete with in-house technicians. Additionally, we schedule each task, specifying its start time and assigned maintenance base. In-house tasks are further assigned to a technician's work sequence. Once task processing times are revealed, we adjust each task start time and flight departure time accordingly to estimate the expected overtime and flight departure costs. (ii) Our heuristic solution is a contribution itself as it extends the ability of our model to find solutions in relatively large-sized instances. We propose a novel approach that combines AILS to search for first-stage solutions with a dynamic programming (DP) model that exactly computes the cost of each scenario to estimate expected second-stage costs. (iii) Moreover, we present significant cost-saving opportunities averaging 74%, 34% and 14% when we empirically compare our methodology to alternative planning strategies that consider average processing times, maximum processing times or an optimized time buffer set between consecutively scheduled tasks, respectively. Additionally, our analysis assesses the benefits of the Line Maintenance planning philosophy, *i.e.*, breaking down maintenance packages into more and less time-consuming tasks. Also, we estimate the cost induced by task processing time uncertainty in Line Maintenance operations.

3. Problem definition

We consider the operation of an airline working with a fleet U of aircraft executing a set of flight itineraries $\mathcal{F} := \{F_1, F_2, \dots, F_{|U|}\}$ over an operating horizon of T time units. We focus on a short-term plan (*e.g.*, a 24-hour day) that follows a medium-term aggregated maintenance schedule, such as weekly or monthly plan. Each itinerary $F_u := \{f_{u,1}, f_{u,2}, \dots, f_{u,q_u}\}$ is a sequence of q_u flight legs performed by a specific aircraft $u \in U$. Each flight leg f is determined by an origin and destination pair of airports, has a scheduled departure time h_f , and a duration τ_f . This duration τ_f is defined as the sum of flight time and a minimum required downtime, which accounts for the time necessary before the aircraft is ready for its next flight or maintenance. Therefore, an aircraft taking off at time h_f is available to fly again at time $h_f + \tau_f$. In practice, flights may depart later than their scheduled time. Delayed departures are undesirable for planners, so we assign a cost $\$a_f$ per unit of delay. This cost reflects both the financial impact on the airline and the loss of customer goodwill due to the delay.

The airline relies on a set A of bases to execute maintenance. In each base $a \in A$, there exists a daily presence of r_a technicians available to work on maintenance tasks. A technician can only execute one task at a given time. Also, maintenance tasks can only be executed within its specific aircraft MOs. Specifically, there exists a set of MOs O , where each $o \in O$ is determined by an available ground time window of a particular flight itinerary. Each MO $o \in O$ occurs between two consecutive flights, except for the first and last MOs of an aircraft that occur between the start of the operation and the first scheduled flight departure and between the last flight arrival and the end of the operation, respectively. Let O_{start} and O_{end} be the sets containing all MOs in O that occur

between the start of the operation and the first flight of an aircraft and between the last flight arrival and the end of the operation, respectively. The pair of flights immediately succeeding and preceding MO $o \in O$ is denoted as $f^+(o)$ and $f^-(o)$, respectively. The arrival time of flight $f^-(o)$ sets the beginning of o , while the departure of $f^+(o)$ sets its end time. $f^-(o)$ and $f^+(o)$ are assumed empty for first and last MOs of an aircraft.

At the start of each day, we assume that the airline has full knowledge of a set I of tasks to execute that day on the fleet. Each task $i \in I$ must be performed on a specific aircraft and demands the time of one technician while it is being executed. We denote its processing time as p_i , which we assume is a random variable. We assume complete knowledge of their probability distribution and expected value $\bar{p}_i$ based on the historical record of processing times for similar tasks. Moreover, each task i can only be executed in a compatible maintenance MO contained in subset $O_i \subset O$. An MO is compatible with task i if it belongs to the task's aircraft and lasts longer than its minimum processing time. Also, define subset $O_i^a \subset O_i$ as all MOs occurring at base a compatible with task i and set $A_i := \{a \in A : O_i^a \neq \emptyset\}$ as the bases where the task may be executed.

3.1. Decisions

At the beginning of the day, the maintenance planner must schedule the execution of each task $i \in I$ and assign it to a base $a \in A_i$ and a compatible MO $o \in O_i^a$ where and when it will be executed. In addition, the planner must decide whether to outsource or not task i . If task i is outsourced, the airline incurs a penalty $\$ \beta_{i,o}$ depending on its assigned MO o . This penalty represents the cost of assigning a task to a third-party maintenance provider and may vary depending on the chosen MO due to potential fluctuations in the external resource availability across different bases and over time. Otherwise, if task i is performed in-house, the planner must schedule its execution into the work sequence of an available technician at base a and MO o .

Once task processing times become known, each task i executed in MO o will start and end at specific times. It will delay flight $f^+(o)$ if its completion time exceeds the flight's scheduled start time $h_{f^+(o)}$. An in-house task can also delay the subsequent scheduled work of its assigned technician at base a if it ends later than expected or if its start time was delayed due to prior disruptions. A specific task may also extend beyond the operating horizon T due to unforeseen delays. In this case, the airline pays overtime at a fixed cost $\$ \mu$ plus a variable cost of $\$ \lambda$ per time unit. Table 9 in Appendix A.1 summarizes all our notation. The objective is to minimize the expected cost of outsourced maintenance tasks, flight delays and overtime.

4. Problem modeling and MIP formulation

In this section, we represent the LMSP with uncertain task processing times as a two-stage stochastic programming model. Section 4.1 introduces the first-stage decisions defining task selection and sequencing. In Section 4.2, we present the second-stage recourse actions, which adjust each task start time and each flight departure time once first-stage decisions are taken and task processing times become known. Finally, Section 4.3 outlines the full two-stage model, detailing how to minimize the combined costs of the first and second stages. We use Monte-Carlo sampling and SAA to estimate the expected second-stage cost.

4.1. First-stage decisions: task assignment and sequencing

Three key decisions are taken in the first-stage model. The first is assigning each task to a maintenance opportunity and is captured in the binary decision vector $\mathbf{x}$, where each component $x_{i,o}$ takes the value one if task $i \in I$ is assigned to MO $o \in O_i$ and 0 otherwise. The second choose which tasks to perform in-house (decision vector $\mathbf{y}$), where each component $y_{i,o}$ takes the value one if task $i \in I$ is performed in-house and assigned to MO $o \in O_i$ and 0 otherwise. The third is sequencing the execution of tasks for each technician resource; it is represented in the binary decision vector $\mathbf{z}$, where each component $z_{i,j}^a$ takes the value one if task $i \in I_0$ immediately precedes task $j \in I_0 \setminus \{i\}$ at base $a \in A_i \cap A_j$ and 0 otherwise. $I_0 := \{I \cup 0\}$ is an extended task set including a fictitious task 0 representing the start/end of each technician's task sequence (also, $A_0 := A$). All tasks not included in vector $\mathbf{y}$ are outsourced and thus vector $\mathbf{y}$ is auxiliary. The joint decision of vector variables $(\mathbf{x}, \mathbf{y}, \mathbf{z})$ constitutes a feasible maintenance plan if it is contained in the first-stage domain $\mathbb{X} := \{(\mathbf{x}, \mathbf{y}, \mathbf{z}) : (1a)-(1h), (2a)-(2d)\}$ defined by the system of integer constraints and linear inequalities in (1) and (2). System (1) imposes basic relations for the first-stage decision variables.

$$\sum_{o \in O_i} x_{i,o} = 1, \quad \forall i \in I \quad (1a)$$

$$y_{i,o} \leq x_{i,o}, \quad \forall i \in I, o \in O_i \quad (1b)$$

$$\sum_{j \in I_0 \setminus \{i\} : a \in A_j} z_{i,j}^a = \sum_{o \in O_i^a} y_{i,o}, \quad \forall i \in I, a \in A_i, \quad (1c)$$

$$\sum_{j \in I_0 \setminus \{i\} : a \in A_j} z_{i,j}^a = \sum_{j \in I_0 \setminus \{i\} : a \in A_j} z_{j,i}^a, \quad \forall i \in I, a \in A_i, \quad (1d)$$

$$\sum_{j \in I : a \in A_j} z_{0,j}^a \leq r_a, \quad \forall a \in A, \quad (1e)$$

$$\sum_{a \in A_i \cap A_j} z_{i,j}^a \leq 2 - y_{i,o} - y_{j,o}, \quad \forall i, j \in I : i > j, o \in O_i \cap O_j, \quad (1f)$$

$$x_{i,o}, y_{i,o} \in \{0, 1\}, \quad \forall i \in I, o \in O_i, \quad (1g)$$

$$z_{i,j}^a \in \{0,1\}, \quad \forall i,j \in I_0 : i \neq j, a \in A_i \cap A_j. \quad (1h)$$

Constraints (1a) guarantee that all tasks are scheduled, while constraints (1b) allow a task to be assigned in-house to MO o only when it is assigned to the same MO. Constraints (1c) relate vectors y and z and guarantee that each in-house executed task i is assigned to a technician sequence and an MO at the same airport. Constraints (1d)–(1e) are technician workflow sequence balance constraints. Constraints (1d) impose that each task $i \in I$ assigned to a technician at base $a \in A_i$ must be preceded and succeeded by another task in a technician's workflow at base a . Constraints (1e) impose a maximum number of simultaneously executed in-house tasks equal to the number of available technicians at base a . Moreover, symmetry-breaking constraints (1f) ensure a lexicographic workflow sequence when two tasks are assigned to the same MO; if two tasks $i, j \in I$ such that $i > j$ are assigned in-house to MO o , then it only allows sequencing task i immediately after j excluding the symmetric assignment where j goes immediately before i . Finally, constraints (1g)–(1h) define the binary nature of our decision variables.

However, only enforcing system (1) may produce maintenance plans having flight delays and overtime for the expected scenario. The latter is undesirable and illegal in some countries that prohibit scheduling overtime for workers (e.g., Chile). System (2) rules out all such incompatible maintenance plans. For each task $i \in I$, it defines a real auxiliary variable $\bar{s}_i \geq 0$ representing its resulting start time if the expected scenario occurs.

$$\bar{s}_i \geq \sum_{o \in O_i \setminus O_{\text{start}}} (h_{f^-(o)} + \tau_{f^-(o)}) \cdot x_{i,o}, \quad \forall i \in I, \quad (2a)$$

$$\bar{s}_i + \bar{p}_i \leq \sum_{o \in O_i \setminus O_{\text{end}}} (h_{f^+(o)} - T) \cdot x_{i,o} + T, \quad \forall i \in I, \quad (2b)$$

$$\bar{s}_i + \bar{p}_i \leq \bar{s}_j + \left(1 - \sum_{a \in A_i \cap A_j} z_{i,j}^a\right) \cdot T, \quad \forall i, j \in I : i \neq j, \quad (2c)$$

$$\bar{s}_i \geq 0, \quad \forall i \in I. \quad (2d)$$

Constraints (2a) impose that each task i assigned to MO o will start after the scheduled landing time of its preceding flight. Constraints (2b) guarantee that each task i concludes, in the expected scenario, before the scheduled departure of its succeeding flight. Finally, (2c) enforce the correct relation between start times of consecutively sequenced tasks.

4.2. Second-stage decisions: task start and flight departure time adjustment

The second stage model sets each task start time and each flight departure time for given values of first-stage decisions $(\mathbf{x}, \mathbf{y}, \mathbf{z}) \in \mathbb{X}$ and realized scenario ω within the set Ω of sampled scenarios. Each scenario ω defines the disclosed processing time p_i^ω for each task $i \in I$.

Let $s_i^{S,\omega} \geq 0$ and $t_f^{D,\omega} \geq 0$ be scenario variables representing the start time of task i and the departure time of flight f in scenario ω . We couple these decisions with additional auxiliary variables: the departure delay of flight f set by variable $t_f^{D,\omega} \geq 0$ and the overtime required to complete task i defined as $w_i^\omega \geq 0$. Also, let binary variable g_i^ω be equal to 1 if task i starts within the operating horizon but ends after it and 0 otherwise, and let binary variable l_i^ω be equal to 1 if task i starts after the end of the operating horizon and 0 otherwise. The second-stage decision is feasible if it is contained in the second-stage domain $\mathbb{Y}(\mathbf{x}, \mathbf{z}, \omega) := \{(s^\omega, \mathbf{t}^{S,\omega}, \mathbf{t}^{D,\omega}, \mathbf{w}^\omega, \mathbf{g}^\omega, \mathbf{l}^\omega) : (3a)–(3m)\}$ defined by the system of integer linear inequalities in (3) for each scenario $\omega \in \Omega$.

$$t_f^{S,\omega} - h_f \leq t_f^{D,\omega}, \quad \forall u \in U, f \in F_u, \quad (3a)$$

$$t_f^{S,\omega} \geq h_f, \quad \forall u \in U, f \in F_u, \quad (3b)$$

$$t_{f_{u,v}+1}^{S,\omega} \geq t_{f_{u,v}}^{S,\omega} + \tau_{f_{u,v}}, \quad u \in U, v \in \{1, \dots, q_u - 1\}, \quad (3c)$$

$$s_i^\omega + (1 - x_{i,o}) \cdot 2T \geq t_{f^-(o)}^{S,\omega} + \tau_{f^-(o)}, \quad \forall i \in I, o \in O_i \setminus O_{\text{start}}, \quad (3d)$$

$$s_i^\omega + p_i^\omega \leq t_{f^+(o)}^{S,\omega} + (1 - x_{i,o}) \cdot 2T, \quad \forall i \in I, o \in O_i \setminus O_{\text{end}}, \quad (3e)$$

$$s_i^\omega + p_i^\omega \leq s_j^\omega + \left(1 - \sum_{a \in A_i \cap A_j} z_{i,j}^a\right) \cdot 2T, \quad \forall i, j \in I : i \neq j, \quad (3f)$$

$$s_i^\omega \leq T \cdot (1 + l_i^\omega), \quad \forall i \in I, \quad (3g)$$

$$s_i^\omega + p_i^\omega \leq T \cdot (1 + l_i^\omega + g_i^\omega), \quad \forall i \in I, \quad (3h)$$

$$w_i^\omega \geq s_i^\omega + p_i^\omega - T \cdot (2 - g_i^\omega), \quad \forall i \in I, \quad (3i)$$

$$w_i^\omega \geq p_i^\omega - T \cdot (1 - l_i^\omega), \quad \forall i \in I, \quad (3j)$$

$$g_i^\omega, l_i^\omega \in \{0,1\}, \quad \forall i \in I, \quad (3k)$$

$$s_i^\omega, w_i^\omega \geq 0, \quad \forall i \in I, \quad (3l)$$

$$t_f^{S,\omega}, t_f^{D,\omega} \geq 0, \quad \forall u \in U, f \in F_u. \quad (3m)$$

Inequalities (3a)–(3c) impose constraints on flight departure times and flight delays. Constraints (3a) restrict delays to be greater than or equal to the difference between realized and scheduled departure times; constraints (3b) prohibit flights to depart earlier than their scheduled departure; and constraints (3c) guarantee that the departure time of each flight to be greater than or equal to the arrival time of the preceding flight plus the minimum required downtime. The set (3d)–(3f) relate each task's start and end times to its assigned MO. Constraints (3d) impose each task i start time to be greater than or equal to the arrival time of the preceding flight $f^-(o)$ of its assigned MO o . Constraints (3e) are analogous and guarantee each task i end time to be less than or equal to the departure time of the succeeding flight $f^+(o)$ of its assigned MO o . Constraints (3f) force each task i to end before the start of its immediately succeeding task j in its assigned technician sequence. Constraints (3g)–(3j) impose overtime requirements. Constraints (3g) trigger overtime binary variable l_i^ω for each task i starting after time T . Constraints (3h) trigger overtime binary variables l_i^ω or g_i^ω for each task i ending after time T . Constraints (3i) impose that overtime required for a task i ending past T to be greater or equal to $s_i^\omega + p_i^\omega - T$. Finally, constraints (3j) are analogous and impose that overtime required for a task i starting past T to be greater or equal to p_i^ω . Note that second-stage decisions are independent of vector y , as the necessary information for in-house scheduled tasks is contained in decision vector z .

4.3. Two-stage model

We now define our two-stage model (4) based on the predefined sets of first and second-stage decisions. Its objective function minimizes the total cost of outsourced tasks plus the expected second-stage cost. Model (5) defines the second-stage cost for each scenario $\omega \in \Omega$ to be equal to the minimum total cost of flight delays plus overtime.

$$\min_{(\mathbf{x}, \mathbf{y}, \mathbf{z}) \in \mathbb{X}} \sum_{i \in I} \sum_{o \in O_i} \beta_{i,o} \cdot (x_{i,o} - y_{i,o}) + \mathbb{E}_\omega[Q(\mathbf{x}, \mathbf{z}, \omega)]. \quad (4)$$

$$Q(\mathbf{x}, \mathbf{z}, \omega) = \min_{(s, t, S, t^D, w, g, l) \in \mathbb{Y}(\mathbf{x}, \mathbf{z}, \omega)} \sum_{f \in F} \alpha_f \cdot t_f^{D,\omega} + \sum_{i \in I} (g_i^\omega + l_i^\omega) \cdot \mu + \lambda \cdot w_i^\omega. \quad (5)$$

5. Solution algorithms

We propose exact and approximate algorithms to solve model (4). Both algorithms explore feasible first-stage decisions $(\mathbf{x}, \mathbf{y}, \mathbf{z}) \in \mathbb{X}$ and evaluate its expected second-stage $Q(\mathbf{x}, \mathbf{z}, \omega)$ cost via an efficient DP engine presented in Section 5.1. Our proposed exact algorithm is a custom version of the Integer L-shaped method and it is described in Section 5.2. Our heuristic algorithm combines the AILS framework with the DP engine and is presented in Section 5.3.

5.1. DP algorithm to compute the second-stage model

For given sampled scenario $\omega \in \Omega$ and first stage solution $(\mathbf{x}, \mathbf{z})$, model (5) is used to define the start times of two types of events: task start times in vector s^ω and flight departure times in vector $t^{S,\omega}$. Also, each event start time value has precedence constraints depending on related events. This dependency structure can be modeled as a directed acyclic graph (DAG) similar to a well-known PERT model (Van Slyke, 1963) and, thus, DP is a natural solution to solve problem (5).

Given $(\mathbf{x}, \mathbf{z}, \omega)$, we define a DAG $G := (N, E)$ where N and E are its sets of nodes and edges, respectively. Set N contains all events with start times in vector π , including flight departure and task start times. Formally define $N := \{F \cup I\}$, where $F := \cup_{u \in U} F_u$ is the set of all flight legs. Each event $k \in N$ also has a minimum feasible start time m_k parameter, which is equal to zero for task start times and equal to h_k in case of flight departure times; these model the family of constraints in (3b).

Set E represents all precedence constraints between pairs of activities imposed in constraints (3c)–(3f) and it is defined as the union of four sets of edges $E := E_{F \rightarrow F} \cup E_{F \rightarrow I} \cup E_{I \rightarrow F} \cup E_{I \rightarrow I}$, where:

- $E_{F \rightarrow F} := \{(f_{u,v}, f_{u,v+1}) | \forall v \in \{1, \dots, q_u - 1\}, u \in U\}$ are flight-to-flight edges representing constraints (3c). We set a duration $\gamma_e := \tau_f$ for each edge $e = (f, f') \in E_{F \rightarrow F}$.
- $E_{F \rightarrow I} := \{(f^-(o), i) | \forall i \in I, o \in O_i \setminus O_{\text{start}} : x_{i,o} = 1\}$ are flight-to-task edges representing constraints (3d). We set a duration $\gamma_e := \tau_f$ for each edge $e = (f, i) \in E_{F \rightarrow I}$.
- $E_{I \rightarrow F} := \{(i, f^+(o)) | \forall i \in I, o \in O_i \setminus O_{\text{end}} : x_{i,o} = 1\}$ are task-to-flight edges representing constraints (3e). We set a duration $\gamma_e := p_i^\omega$ for each edge $e = (i, f) \in E_{I \rightarrow F}$.
- $E_{I \rightarrow I} := \{(i, j) | \forall i, j \in I : i \neq j, \sum_a z_{i,j}^a = 1\}$ are task-to-task edges representing constraints (3f). We set a duration $\gamma_e := p_i^\omega$ for each edge $e = (i, j) \in E_{I \rightarrow I}$.

Also, let $N_k^- := \{k' \in N : (k', k) \in E\}$ be the set of nodes preceding node $k \in N$.

Fig. 3 depicts an example of graph G for the particular instance plan depicted on Fig. 1 with only one aircraft executing flights $F = (f_1, f_2)$ and one technician sequence $(1, 3, 4)$ of three tasks. Task 1, 2 and 3 are assigned in the MO before f_1 , and Task 4 is assigned to the MO between both flights.

In this graph G , an optimal vector π^* of event start times can be obtained by setting the start time π_k of each event $k \in N$ as early as possible following a topological order over G . Formally, let $\{1, \dots, n\}$ be the set indexes of nodes in G such that it

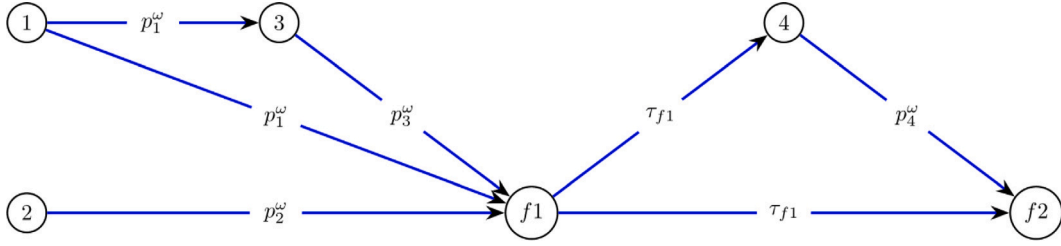
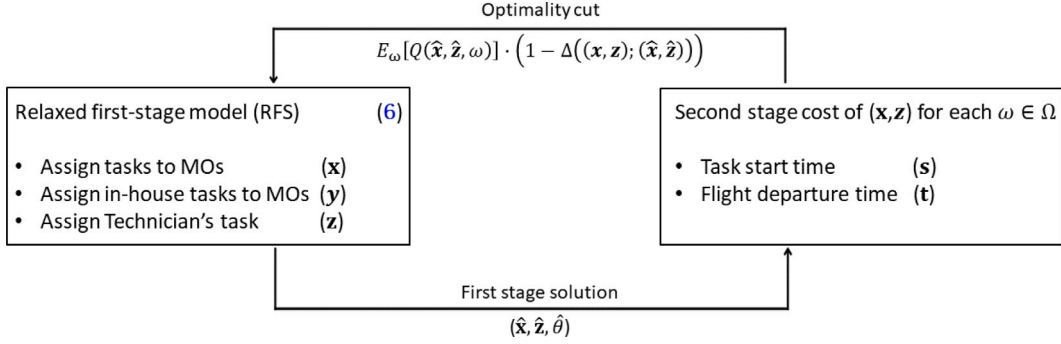
Fig. 3. Example of event DAG G for the example in Fig. 1.

Fig. 4. Scheme of the proposed two-stage solution algorithm.

follows a topological order. The DP defined in Algorithm 1 finds the optimal cost of second-stage model (5) for a given sampled scenario $\omega \in \Omega$ and feasible first stage solution $(\mathbf{x}, \mathbf{z})$. Moreover, the algorithm has $\mathcal{O}(E)$ running time complexity, and therefore, the expected optimal second stage cost can be evaluated in $\mathcal{O}(E \times |\Omega|)$ running time complexity. The proof of correctness of Algorithm 1 is straightforward: each event cost c is a non-decreasing function of π_k for each event $k \in N$, and the assignment of vector π in topological order over G sets each value of π_k to its minimum feasible value.

Algorithm 1 Optimal second-stage cost DP

Require: $\mathbf{x}, \mathbf{z}, \omega$

- 1: $C \leftarrow 0$
 - 2: Build graph $G = (N, E)$ and find a topological order $\{1, \dots, n\}$ for it.
 - 3: Initialize the start time vector $\pi \leftarrow \{\}$
 - 4: **for** $r \in \{1, \dots, n\}$ **do**
 - 5: $\pi_{k(r)} = \max\{m_k, \max_{k' \in N_k^-} (\pi_{k'} + \gamma_{k',k})\}$
 - 6: **if** $k \in F$ **then**
 - 7: **if** $\pi_k > h_k$ (there is flight delay) **then**
 - 8: Set cost $c \leftarrow \alpha_k \cdot (\pi_k - h_k)$
 - 9: **if** $k \in I$ **then**
 - 10: **if** $\pi_k + p_k^\omega > T$ (task requires overtime) **then**
 - 11: Set cost $c \leftarrow \mu + \lambda \cdot (\pi_k + p_k^\omega - \max\{\pi_k, T\})$
 - 12: $C \leftarrow C + c$
 - return** C
-

5.2. Exact solution: Integer L-shaped method

Due to the large number of scenarios and task selection and sequencing decisions involved, it is computationally intractable to solve the extended formulation resulting from the combined models (4) and (5). Consequently, we present a Benders type decomposition scheme (Benders, 1962) commonly referred to as the Integer L-shaped method (Laporte and Louveaux, 1993). Fig. 4 describes a diagram of the proposed two-stage solution method. Specifically, we iteratively solve a relaxed version of the first-stage model (4) referred as RFS (6) in which we replace the expected second-stage cost with a non-negative estimate θ satisfying

$\mathbb{E}_\omega[Q(\mathbf{x}, \mathbf{z}, \omega)] \geq \theta \geq 0$ for each $(\mathbf{x}, \mathbf{z}) \in \mathbb{X}$.

$$\min_{(\mathbf{x}, \mathbf{y}, \mathbf{z}) \in \mathbb{X}, \theta \geq 0} \left\{ \sum_{i \in I} \sum_{o \in O_i} \beta_{i,o} \cdot (x_{i,o} - y_{i,o}) + \theta \right\} \quad (6)$$

We run a single Branch & Bound (B&B) tree Benders type algorithm over the RFS model defined in (6) until we achieve zero optimality gap or meet timeout conditions. Each B&B node is pruned if its optimal relaxed LP cost is greater than or equal to the exact cost of the incumbent. If a new integer solution $(\hat{\mathbf{x}}, \hat{\mathbf{z}}, \hat{\theta})$ for the RFS model is found, then we compute its exact expected second-stage cost $\mathbb{E}_\omega[Q(\hat{\mathbf{x}}, \hat{\mathbf{z}}, \omega)]$ by exactly solving the second-stage model (5) for each scenario $\omega \in \Omega$ via the DP Algorithm 1. If $\hat{\theta} < \mathbb{E}_\omega[Q(\hat{\mathbf{x}}, \hat{\mathbf{z}}, \omega)]$, then we adjust our relaxed model (6) adding the following optimality cut to its feasible first-stage domain

$$\theta \geq \mathbb{E}_\omega[Q(\hat{\mathbf{x}}, \hat{\mathbf{z}}, \omega)] \cdot (1 - \Delta((\mathbf{x}, \mathbf{z}); (\hat{\mathbf{x}}, \hat{\mathbf{z}}))), \quad (7)$$

that cuts-off each feasible solution $(\hat{\mathbf{x}}, \hat{\mathbf{z}}, \hat{\theta})$ from model (6), where

$$\Delta((\mathbf{x}, \mathbf{z}); (\hat{\mathbf{x}}, \hat{\mathbf{z}})) := \sum_{(i,o) \in S(\hat{\mathbf{x}})} (1 - x_{i,o}) + \sum_{(i,o) \notin S(\hat{\mathbf{x}})} x_{i,o} + \sum_{(i,j,a) \in S(\hat{\mathbf{z}})} (1 - z_{i,j}^a) + \sum_{(i,j,a) \notin S(\hat{\mathbf{z}})} z_{i,j}^a \quad (8)$$

is the Hamming distance between the binary vectors $(\hat{\mathbf{x}}, \hat{\mathbf{z}})$ and $(\mathbf{x}, \mathbf{z})$ and sets $S(\hat{\mathbf{x}})$, and $S(\hat{\mathbf{z}})$ are the cardinality sets of $\hat{\mathbf{x}}$ and $\hat{\mathbf{z}}$, respectively. Formally, $S(\hat{\mathbf{x}}) := \{(i, o) : i \in I, o \in O_i, \hat{x}_{i,o} = 1\}$ and $S(\hat{\mathbf{z}}) := \{(i, j, a) : i, j \in I_0, i \neq j, a \in A_i \cap A_j, \hat{z}_{i,j}^a = 1\}$. If $\hat{\theta} \geq \mathbb{E}_\omega[Q(\hat{\mathbf{x}}, \hat{\mathbf{z}}, \omega)]$, then solution $(\hat{\mathbf{x}}, \hat{\mathbf{z}}, \hat{\theta})$ is feasible to the exact first-stage model (4) and we update the incumbent if this solution is cheaper. Algorithm 2 summarizes our exact solution method in pseudo-code.

Algorithm 2 Exact solution algorithm

```

1: while gap > 0 and not timeout do
2:   Obtain the next open node of (6) in the B&B tree.
3:   Apply node pruning and branching rules.
4:   if B&B node yields an integer solution  $(\hat{\mathbf{x}}, \hat{\mathbf{z}})$  then
5:     for  $\omega \in \Omega$  do
6:       Compute  $Q(\hat{\mathbf{x}}, \hat{\mathbf{z}}, \omega)$  via DP
7:       Estimate  $\mathbb{E}_\omega[Q(\hat{\mathbf{x}}, \hat{\mathbf{z}}, \omega)] \approx \frac{1}{|\Omega|} \sum_{\omega \in \Omega} Q(\hat{\mathbf{x}}, \hat{\mathbf{z}}, \omega)$ 
8:       if  $\hat{\theta} < \mathbb{E}_\omega[Q(\hat{\mathbf{x}}, \hat{\mathbf{z}}, \omega)]$  then
9:         Add optimality cut (7) to model (6).
10:      else
11:        Try incumbent update.
12:      Update gap.
13: return Best solution found

```

Property 1 (Non-Decreasing Second-Stage Cost). Let $(\hat{\mathbf{x}}, \hat{\mathbf{y}}, \hat{\mathbf{z}})$ represent a feasible first-stage solution to model (4), and let $Q(\hat{\mathbf{x}}, \hat{\mathbf{z}}, \omega)$ denote its second-stage cost for a given scenario $\omega \in \Omega$. If we include more tasks at the start or end of the current task in-house sequence $\mathbf{z}$ without altering the previous sequence and MOs assignments, the new solution $(\mathbf{x}', \mathbf{y}', \mathbf{z}')$ will have a second-stage cost $Q(\mathbf{x}', \mathbf{z}', \omega) \geq Q(\hat{\mathbf{x}}, \hat{\mathbf{z}}, \omega)$

Proof. We can express the cost for the new solution, $Q(\mathbf{x}', \mathbf{z}', \omega)$, as

$$Q(\mathbf{x}', \mathbf{z}', \omega) = Q(\hat{\mathbf{x}}, \hat{\mathbf{z}}, \omega) + \Delta_{\text{new}} + \Delta_{\text{current}}, \quad (9)$$

where Δ_{new} represents the direct cost of newly added in-house tasks and Δ_{current} accounts for the change in cost due to potential delays in previous activities. Specifically, Δ_{new} is the sum of the individual direct costs of each newly added task. Each of these individual costs is non-negative, ensuring that Δ_{new} is also non-negative. The start time of the activities relies on the dependencies represented in the directed acyclic graph. This graph dictates the earliest possible start times for both tasks and flights. By maintaining the MO assignments and the previous sequence, we preserve the lower bounds. Consequently, when we add new tasks at the end of the sequence, they do not alter the start times or costs of current activities, leading to $\Delta_{\text{current}} = 0$. Conversely, adding new tasks at the start of the sequence can delay the start times of subsequent tasks or flights. Therefore, Δ_{current} is non-negative. $\square$

We further improve the optimality cut used in our exact solution algorithm and add an enhanced version defined in Property 2.

Property 2 (Enhanced Optimality Cut). The enhanced optimality cut defined as

$$\theta \geq \mathbb{E}_\omega[Q(\hat{\mathbf{x}}, \hat{\mathbf{z}}, \hat{\theta})] \cdot (1 - \Delta'((\mathbf{x}, \mathbf{z}); (\hat{\mathbf{x}}, \hat{\mathbf{z}}))), \quad (10)$$

where

$$\Delta'((\mathbf{x}, \mathbf{z}); (\hat{\mathbf{x}}, \hat{\mathbf{z}})) = \sum_{(i,o) \in S(\hat{\mathbf{x}})} (1 - x_{i,o}) + \sum_{(i,j,a) \in S(\hat{\mathbf{z}})} (1 - z_{i,j}^a), \quad (11)$$

and $\bar{S}(\hat{z}) := \{(i, j, a) : i, j \in I, i \neq j, a \in A_i \cap A_j, \hat{z}_{i,j}^a = 1\}$ is valid for model (6) at any stage of the integer L-shaped method.

Proof. The cut keeps a valid lower bound θ of the expected second-stage cost for each feasible value $(\mathbf{x}, \mathbf{z})$.

- If $(\mathbf{x}, \mathbf{z}) = (\hat{\mathbf{x}}, \hat{\mathbf{z}})$, then $\Delta'((\mathbf{x}, \mathbf{z}); (\hat{\mathbf{x}}, \hat{\mathbf{z}})) = 0$ and the enhanced cut yields $\theta \geq \mathbb{E}_\omega[Q(\hat{\mathbf{x}}, \hat{\mathbf{z}}, \omega)]$. This keeps θ as a valid lower bound for $\mathbb{E}_\omega[Q(\mathbf{x}, \omega)]$.
- If $(\mathbf{x}, \mathbf{z}) \neq (\hat{\mathbf{x}}, \hat{\mathbf{z}})$ and $\Delta'((\mathbf{x}, \mathbf{z}); (\hat{\mathbf{x}}, \hat{\mathbf{z}})) = 0$, then the enhanced cut also yields $\theta \geq \mathbb{E}_\omega[Q(\hat{\mathbf{x}}, \hat{\mathbf{z}}, \omega)]$. Also, solution $(\mathbf{x}, \mathbf{z})$ covers at least the selected set and sequence of in-house tasks as in solution $(\hat{\mathbf{x}}, \hat{\mathbf{z}})$. According to Property 1, the second-stage cost $\mathbb{E}_\omega[Q(\hat{\mathbf{x}}, \hat{\mathbf{z}}, \omega)]$ is less than or equal to $\mathbb{E}_\omega[Q(\mathbf{x}, \mathbf{z}, \omega)]$. Therefore θ is still a valid lower bound for $\mathbb{E}_\omega[Q(\mathbf{x}, \mathbf{z}, \omega)]$.
- Finally, if $(\mathbf{x}, \mathbf{z}) \neq (\hat{\mathbf{x}}, \hat{\mathbf{z}})$ and $\Delta'((\mathbf{x}, \mathbf{z}); (\hat{\mathbf{x}}, \hat{\mathbf{z}})) \geq 1$, then the enhanced cut yields $\theta \geq \alpha$, where $\alpha \leq 0$ and thus is a valid lower bound. $\square$

5.3. Heuristic solution: Adaptive iterated local search

We now present a heuristic solution to Model (4) based on AILS to solve large-sized instances. Similar to our exact solution algorithm, we use the DP algorithm defined in Section 5.1 to evaluate the cost of each solution $(\mathbf{x}, \mathbf{y}, \mathbf{z}) \in \mathbb{X}$. Given a solution, AILS tries to identify a cheaper one by applying a series of local search and random perturbations moves to it. To refine the search, AILS also dynamically adapts the selection probability of each local search operator, random perturbation operator, and perturbation intensity. Distinctly, we apply filters to rule out any infeasible or inefficient candidate solution.

Algorithm 3 presents the skeleton of our heuristic in pseudocode. We start from an existing feasible solution improved via local search, which is saved as the incumbent in line 2. We also initialize adaptive parameters, Φ_{LS} and Φ_{RP} , which store the performance of the local search operators and the random perturbation types, respectively. Additionally, we start a counter that keeps track of the number of iterations performed without updating the incumbent. Then, we enter the loop of iterations. In each cycle, we run adaptive random perturbation and local search algorithms over the last solution found in lines 4 and 5; we also update the adaptive parameters each time these algorithms are called. Finally, we update the incumbent in line 8 and reset the iteration counter in line 9 each time we find a cheaper solution. We keep refining the incumbent until we reach a predefined maximum number of iterations without improvement $N^{\max}$ or a time limit $T^{\max}$. The initial solution required in Algorithm 3 is constructed by solving a deterministic version of model (4) for just one average scenario $\bar{\omega}$ and a maximum computation time of 300 s. More details of the local search and random perturbation algorithms are found in Sections 5.3.1 and 5.3.2, respectively.

Algorithm 3 Adaptive Iterated Local Search (AILS)

Require: Solution $(\mathbf{x}, \mathbf{y}, \mathbf{z})$, max. iterations $N^{\max}$ without improvement, and time limit $T^{\max}$.

```

1: Initialize adaptive parameters and counter:  $\Phi_{LS}, \Phi_{RP}, n \leftarrow 0$ 
2: Initialize incumbent:  $(\mathbf{x}^*, \mathbf{y}^*, \mathbf{z}^*), C^*, \Phi_{LS} \leftarrow \text{LocalSearch}(\mathbf{x}, \mathbf{y}, \mathbf{z}, \Phi_{LS})$ 
3: while  $n < N^{\max}$  and  $\text{time} < T^{\max}$  do
4:    $(\mathbf{x}, \mathbf{y}, \mathbf{z}), \Phi_{RP} \leftarrow \text{RandomPerturbation}(\mathbf{x}, \mathbf{y}, \mathbf{z}, \Phi_{RP}, n)$ 
5:    $(\mathbf{x}, \mathbf{y}, \mathbf{z}), C, \Phi_{LS} \leftarrow \text{LocalSearch}(\mathbf{x}, \mathbf{y}, \mathbf{z}, \Phi_{LS})$ 
6:    $n \leftarrow n + 1$ 
7:   if  $C < C^*$  then
8:     Update incumbent:  $(\mathbf{x}^*, \mathbf{y}^*, \mathbf{z}^*) \leftarrow (\mathbf{x}, \mathbf{y}, \mathbf{z}), C^* \leftarrow C$ 
9:     Reset counter:  $n \leftarrow 0$ 
10: return  $(\mathbf{x}^*, \mathbf{y}^*, \mathbf{z}^*)$ 

```

5.3.1. Local search

Our local search procedure is detailed in pseudocode in Algorithm 4. The process begins with an initial solution from which an incumbent is set in line 2. Then, we initialize the set O_{LS} of local search moves in line 3 with the following standard moves over the vector of task sequences $\mathbf{z}$ (see Groër et al. (2010), McNabb et al. (2015) for more detail on these moves). (1) Inject a task in a given sequence position; (2) remove a task in a given sequence position; (3) swap two tasks within the sequence; (4) remove a task in a given sequence position and replace it with one outsourced task; (5) remove two tasks in a given sequence position and replace them with one outsourced task; (6) remove a task in a given sequence position and replace it with two outsourced tasks; and (7) relocate a task from one sequence position to another. Fig. 5 presents an example of vector $\mathbf{z}$ and a swap move between tasks i and j . Modifying $\mathbf{z}$ to $\hat{\mathbf{z}}$ may produce an infeasible trio $(\mathbf{x}, \hat{\mathbf{y}}, \hat{\mathbf{z}})$ and thus we require to update $\mathbf{x}$ and $\mathbf{y}$. Identifying a feasible couple $(\hat{\mathbf{x}}, \hat{\mathbf{y}})$ for $\hat{\mathbf{z}}$ is complex due to the potentially large number of options. Therefore, we restrict our search and try a fast recovery via a fast exploration over $(\mathbf{x}, \mathbf{y})$. We explore the potential vectors $(\hat{\mathbf{x}}, \hat{\mathbf{y}})$ from $(\mathbf{x}, \mathbf{y})$, focusing only on the MOs impacted by the latest movement in $\mathbf{z}$. Specifically, each $(\hat{\mathbf{x}}, \hat{\mathbf{y}})$ is derived as a copy of $(\mathbf{x}, \mathbf{y})$, varying only the MOs for the affected tasks while keeping all other tasks unchanged. If this approach generates a new candidate solution $(\hat{\mathbf{x}}, \hat{\mathbf{y}}, \hat{\mathbf{z}})$, then we enter the second while loop as reported in line 6. As outlined in line 7, we apply several filters to each candidate solution $(\hat{\mathbf{x}}, \hat{\mathbf{y}}, \hat{\mathbf{z}})$ that quickly discard infeasible or inefficient candidates and reduce the number of second-stage cost evaluations. These are defined as follows. (1) Cycle filter, which discards infeasible

Algorithm 4 Local Search

Require: Solution (x, y, z) , Φ_{LS}

- 1: Current cost: $C \leftarrow \text{Cost}(x, y, z)$
- 2: Initialize incumbent: $(x^*, y^*, z^*) \leftarrow (x, y, z)$, $C^* \leftarrow C$
- 3: Fill set O_{LS} with local search moves
- 4: $\Phi_{LS}, S_{LS} \leftarrow \text{Update}_{LS}(\Phi_{LS}, O_{LS})$
- 5: **while** $O_{LS} \neq \emptyset$ **do**
- 6: **while** A new candidate $(\hat{x}, \hat{y}, \hat{z})$ is found when applying move S_{LS} to (x, y, z) **do**
- 7: **if** $(\hat{x}, \hat{y}, \hat{z})$ fails a filter **then**
- 8: **Break**
- 9: $\hat{C} \leftarrow \text{Cost}(\hat{x}, \hat{y}, \hat{z}, C^*)$
- 10: **if** $\hat{C} < C^*$ **then**
- 11: Update incumbent: $(x^*, y^*, z^*) \leftarrow (\hat{x}, \hat{y}, \hat{z})$, $C^* \leftarrow \hat{C}$
- 12: Fill set O_{LS} with local search move
- 13: **if** $\hat{C} < C \cdot (1 - \epsilon)$ **then**
- 14: Update solution: $(x, y, z) \leftarrow (x^*, y^*, z^*)$, $C \leftarrow C^*$
- 15: **if** $(x^*, y^*, z^*) \neq (x, y, z)$ **then**
- 16: Update solution: $(x, y, z) \leftarrow (x^*, y^*, z^*)$, $C \leftarrow C^*$
- 17: **else**
- 18: $O_{LS} \leftarrow O_{LS} \setminus \{S_{LS}\}$
- 19: $\Phi_{LS}, S_{LS} \leftarrow \text{Update}_{LS}(\Phi_{LS}, O_{LS})$
- 20: **return** $(x^*, y^*, z^*), C^*, \Phi_{LS}$

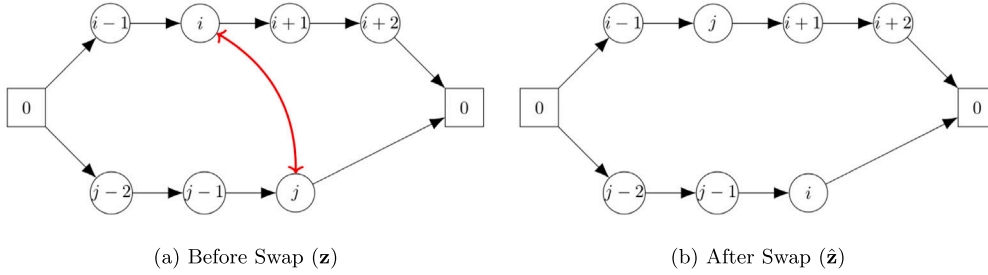


Fig. 5. Example of a swap move.

candidates with cycles in its graph G resulting from the local search move; (2) average scenario feasibility filter, which discards infeasible candidates violating system (2); (3) inefficiency filter, which discards suboptimal candidates that outsource tasks and have zero second-stage cost; and (4) lower bound filter, which discards suboptimal candidates more expensive than the incumbent based on lower bound $\sum_{i \in I} \sum_{o \in O_i} \beta_{i,o} \cdot (\hat{x}_{i,o} - \hat{y}_{i,o}) + \mathbb{E}_\omega \{(Q_{LB}(\hat{x}, \hat{z}, \omega))\}$, where $Q_{LB}(\hat{x}, \hat{z}, \omega)$ is computed assuming that flights are not delayed by previous flight delays (i.e., it discards propagated flight delays). Then, the exact cost $\hat{C}$ of the candidate solution $(\hat{x}, \hat{y}, \hat{z})$ is computed in line 9 calling function $\text{Cost}(\cdot)$ defined in Algorithm 5. If $\hat{C}$ is cheaper than C^* , we update the incumbent in line 11, reset the set of local search moves to explore in line 12 and update the following solution to explore when the produced percentage savings are more significant than 100 $\epsilon\%$ in line 13. The latter balances 'next improvement' and 'best improvement' strategies to update the next solution. Instead of taking the first improvement found, we check if the improvement is significant. If we explore the whole neighborhood of the current local search move without updating $(\hat{x}, \hat{y}, \hat{z})$, we discard this move, switch to another move, update the local search adaptive parameters in line 19, and start a new iteration with a different local search move. The local search algorithm terminates when all local search moves are discarded.

The cost computation of a candidate solution should be fast. Therefore, we speed it up by sorting the set of second-stage scenarios in descending order of their total processing time $\sum_{i \in I} p_i^o$. Let Ω' be this sorted set. Initially, in Algorithm 5 we compute the first-stage cost (line 1). Subsequently, we iterate over each scenario in the sorted set Ω' . If the partial cost C satisfies $C > C^*$ (i.e., it is more expensive than the incumbent), we prematurely stop and return. Otherwise, we compute the scenario's second-stage cost in (line 5) and add it to the partial cost. This routine first evaluates the second-stage cost of scenarios with the highest total processing time, that is, those with probably the most expensive cost. Thus, we may avoid calculating the second stage cost for each scenario in sample Ω .

5.3.2. Random perturbation

Our heuristic also randomly removes (i.e., outsources) a set of scheduled tasks in a given solution (x, y, z) to generate new solutions and escape from local optima. Algorithm 6 summarizes this procedure. We start updating the adaptive parameters, setting

Algorithm 5 Cost

Require: $(\hat{x}, \hat{y}, \hat{z}), C^*$

- 1: $C \leftarrow \sum_{i \in I} \sum_{o \in O_i} \beta_{i,o} \cdot (\hat{x}_{i,o} - \hat{y}_{i,o})$
- 2: **for** $\omega \in \Omega'$ **do**
- 3: **if** $C > C^*$ **then**
- 4: **return** C
- 5: Compute $Q(\hat{x}, \hat{z}, \omega)$ via Algorithm 1
- 6: $C \leftarrow C + \frac{1}{|\Omega|} \cdot Q(\hat{x}, \hat{z}, \omega)$
- 7: **return** C

the number of tasks to remove and outsource $p^{\max}$ and selecting the type of perturbation S_{RP} in line 1. The chosen perturbation type is selected from the following list. (1) Random removal of a scheduled task with equal probability; (2) random removal of a scheduled task with probabilities proportional to average processing time; (3) random removal of a scheduled task with equal probability and its predecessor and successor tasks in the vector of task sequences $\mathbf{z}$; and (4) removal based on lowest task slack time, i.e., the additional time it can last before causing delay or overtime. Then we copy the current solution in line 2 and remove $p^{\max}$ tasks from it in the loop of line 3. The modified copied solution is then returned at the end of the algorithm.

Algorithm 6 Random Perturbation

Require: Solution $(\mathbf{x}, \mathbf{y}, \mathbf{z})$, adaptive parameters Φ_{RP} and counter n

- 1: $\Phi_{RP}, p^{\max}, S_{RP} \leftarrow \text{Update}_{RP}(\Phi_{RP}, n)$
- 2: Copy solution $(\mathbf{x}, \mathbf{y}, \mathbf{z})$ to $(\hat{x}, \hat{y}, \hat{z})$
- 3: **for** $\{1 \dots, p^{\max}\}$ **do**
- 4: Remove a scheduled task i in $(\hat{x}, \hat{y}, \hat{z})$ based on S_{RP} perturbation type
- 5: **return** $(\hat{x}, \hat{y}, \hat{z})$

5.3.3. Adaptive mechanisms

Our heuristic uses online learning mechanisms to adapt the local search and the random perturbation, as a function of the adaptive parameters Φ_{LS} and Φ_{RP} , respectively. The details of this adjustment for local search parameters are outlined in Algorithm 7. We tune the local search move selection probabilities based on historical performance. The performance of each local search operator is measured as the ratio of improvement to computation time, stored in Φ_{LS} . At the beginning of the algorithm, all operators have the same performance value. However, after each iteration, we update the performance of the last employed operator based on how much it improved the solution and the time it expended (line 1). Then, we set all move selection probabilities to zero and later set the selection probability of remaining moves in set O_{LS} to positive values according to a multinomial logit model (line 4); this increases the likelihood of choosing a high-performing move. We then select a local search move based on these probabilities in line 5.

Algorithm 7 Update_{LS}

Require: Φ_{LS}, O_{LS}

- 1: Update Φ_{LS} with the improvement and computing time of the last operator
- 2: Initialize P_{LS} as a vector of zeros.
- 3: **for** $o \in O_{LS}$ **do**
- 4: $P_{LS}(o) = e^{\Phi_{LS}(o)} / \sum_{o' \in O_{LS}} e^{\Phi_{LS}(o')}$
- 5: Select a local search move $S_{LS} \leftarrow \text{Select}(P_{LS})$
- 6: **return** Φ_{LS}, S_{LS}

We apply a similar adjustment strategy for the random perturbation adaptive parameters in Algorithm 8. We record the success rate of each type of perturbation, which is stored in Φ_{RP} . This success rate is calculated as the ratio of improvements to the total number of times each type of perturbation has been employed. Initially, all perturbations start with a default success rate of 1. After each use, we update the success rate of each perturbation type based on its effectiveness (line 1). Subsequently, we update the random perturbation type selection probabilities analogous to the previous algorithm. Additionally, we determine the perturbation intensity $p^{\max}$, which increases with the number of iterations n without improvement. The intensity update is performed at line 5, starting with one-third of the assigned tasks and gradually increasing until outsourcing all assigned tasks in the last possible iteration without improvement, when $n = 9$.

Algorithm 8 Update_{RP}**Require:** Φ_{RP}, n 1: Update Φ_{RP} based on the effectiveness of the last kick2: **for** $r \in$ Perturbation Types **do**3: $P_{RP}(r) = e^{\Phi_{RP}(r)} / \sum_{r' \in R} e^{\Phi_{RP}(r')}$ $\triangleright R$: Set of random kicks4: State $S_{RP} \leftarrow \text{Select}(P_{RP})$ 5: Set intensity $p^{\max}$ as a function of the iteration count n 6: **return** $\Phi_{RP}, p^{\max}, S_{RP}$ **Table 1**

Average number of daily aircraft visits and ground times per aircraft-day at each base.

base	aircraft visits	ground time (hours)
BOG	10.0	4.6
UIO	3.2	9.9
CLO	3.0	5.2
MDE	1.8	6.0

6. Case study

We now present a case study to empirically validate our proposed solution and identify managerial insights. Section 6.1 presents the experimental design, including instance design, benchmark policies used, and KPIs measured. Subsequently, we present and discuss our numerical results in 6.2.

6.1. Experimental design

We created instances representing a 24-hour day operation of a typical airline. In each day's operation, we used real flight itineraries of the domestic fleet of Avianca Airlines, which was formed only by A320FAM aircraft. In this day, an aircraft executes 5–6 flights and produces 2–4 MOs lasting more than one hour. We select flight itineraries from May 2023 employing the pyflightdata API (Allamraju, 2021). First, we extract tail numbers of Avianca's domestic fleet and collect detailed flight information for each tail number, such as date, departure time, flight duration, and origin and destination airports. With this information, we build a test bed of itineraries covering 43 South American destinations, mainly in Colombia and Ecuador. Each operating day starts at noon and concludes 24 h later. We chose this timeline instead of a conventional day to accommodate maintenance working hours since most aircraft remain on the ground and are maintained at night. We set Bogotá (BOG), Quito (UIO), Cali (CLO), and Medellín (MDE) airports as maintenance bases due to their relatively higher average monthly aircraft ground times. Table 1 displays the average number of daily aircraft visits for each base and the average ground time (in hours) per aircraft-day.

Additionally, we set a cost structure that incurs the highest cost in flight delays, followed by overtime and outsourced tasks. Particularly, flight delays cost \$75 per minute as stated in Cook et al. (2004). Overtime costs \$5 per minute plus a fixed cost of \$150 for each event. Finally, we assume time-independent outsourcing costs $\beta_{i,o} = \beta_i = \$5 \cdot \bar{p}_i$, where $\bar{p}_i$ is the task's average processing time in minutes. This approach underlines our operational preference: if a task cannot be executed within its scheduled operational horizon, we tend to outsource it at the start of the operating horizon rather than incurring in overtime.

We define instance classes, each containing multiple instances. Each class is encoded in five structural parameters $(\mathcal{A}, \mathcal{T}, \mathcal{W}, \mathcal{N}, \mathcal{CV})$, representing its number of aircraft ($\mathcal{A}$), number of available technicians at any moment of the day ($\mathcal{T}$), average maintenance workload per aircraft in hours ($\mathcal{W}$), number of maintenance tasks per aircraft ($\mathcal{N}$), and coefficient of variability of task processing times ($\mathcal{CV}$). Table 2 details the employed values for each fixed and class-dependent parameter.

For each value of $\mathcal{T}$, we allocate technicians at bases proportional to available ground times but ensure that at least one technician is assigned per base as presented in Table 3. Also, each instance class has a different size and congestion, defined as the ratio between the total maintenance workload ($\mathcal{A} \cdot \mathcal{W}$) and the total technician effective supply (TES)

$$\text{Congestion} = \frac{\mathcal{A} \cdot \mathcal{W}}{TES}, \quad (12)$$

where TES is defined as the total number of effective technician hours available. In our base experiment, we set congestion levels ranging between 50% and 75%, which is produced by choosing 4 options of parameter vector $(\mathcal{A}, \mathcal{T}, \mathcal{W}) \in \{(10, 4, 4), (15, 6, 4), (20, 4, 2), (20, 8, 4)\}$. Each option is combined with three possible values of $\mathcal{N}$ and three values of $\mathcal{CV}$ obtaining 48 instance classes. For each instance class, we also chose five days of flight itineraries. Also, we generate ten different task processing time distribution settings. The combination of 5 days and 10 settings produces 50 instances per class, resulting in 2400 instances.

We set a triangular processing time distribution for each task. In each case, we set average processing times $\bar{p}_i$ uniformly chosen between $(0.67) \cdot \mathcal{W}/\mathcal{N}$ and $(1.33) \cdot \mathcal{W}/\mathcal{N}$. The minimum, mode, and maximum values are set to $\bar{p}_i \cdot (1 - \epsilon)$, $\bar{p}_i \cdot (1 + 0.5 \cdot \epsilon)$ and $\bar{p}_i \cdot (1 + 1.5 \cdot \epsilon)$, respectively. Parameter ϵ is set equal to $1.852 \cdot \mathcal{CV}$ to yield a triangular distribution with a coefficient of variability equal to $\mathcal{CV}$. A triangular distribution enables a realistic representation of asymmetric processing time distributions and prevents

Table 2
Fixed and class-dependent parameters.

Fixed parameters	Value
operating horizon length (T)	24 hours
maintenance bases (A)	{BOG;UIO;CLO;MED}
cost of outsourcing task i ($\bar{p}_i$)	$\$5 \cdot \bar{p}_i$
delay cost for flight f (α_f)	$\$75$ per minute
fixed overtime cost (μ)	$\$150$
Variable overtime cost (λ)	$\$5$ per minute
Instance class-dependent parameters	Possible values
aircraft (#) ($\mathcal{A}$)	10, 15 or 20
available technicians (#) ($\mathcal{T}$)	4, 6 or 8
workload per aircraft (hours) ($\mathcal{W}$)	2, 4 or 6
maintenance tasks per aircraft (#) ($\mathcal{N}$)	1, 3 or 5
coefficient of variability (CV)	0, 0.1, 0.25 or 0.5

Table 3
Technicians per base as function of $\mathcal{T}$.

$\mathcal{T}$	BOG	UIO	CLO	MDE
4	1	1	1	1
6	3	1	1	1
8	4	2	1	1

Table 4
List of KPIs computed for each instance and solution algorithm.

KPI	Definition
Cost [k\$]	Objective of model (4)
Outsourcing cost [k\$]	$\sum_{i \in I} \sum_{o \in O_i} \beta_{i,o} \cdot (x_{i,o} - y_{i,o})$
Expected Flight delay cost [k\$]	$\sum_{\omega \in \Omega} \frac{1}{ \Omega } \cdot \left(\sum_{f \in F} \alpha_f \cdot t_f^{D,\omega} \right)$
Expected Overtime cost [k\$]	$\sum_{\omega \in \Omega} \frac{1}{ \Omega } \cdot \left(\sum_{i \in I} (g_i^{\omega} + l_i^{\omega}) \cdot \mu + \lambda \cdot u_i^{\omega} \right)$
Outsourced task [%]	Percentage of outsourced tasks
Outsourced task duration [min]	Average duration over outsourced tasks
MOs used [#]	Total number of MOs with scheduled work
Pre-flight buffer [min]	Shortest time between task completion and the next flight departure over all tasks
Technician utilization [%]	Percentage of time used in maintenance operations over the total number of available in-house technician hours at available MOs.
Ground time utilization [%]	Percentage of time used in maintenance operations over the fleet's total ground time at available MOs.

the existence of negative, zero, or excessively longer processing times.

Finally, we generate a random sample of $|\Omega| = 500$ scenarios for each instance. In each scenario $\omega \in \Omega$, we independently simulate a value for each task's processing time realization based on its specific probability distribution.

6.1.1. Benchmark solutions

For each instance, we compare our results to three benchmark solutions. Each benchmark solves a deterministic version of the LMSP model (4) over just one scenario. Then, we compute the exact expected cost of this specific solution in a stochastic environment. The scenario used depends on the specific benchmark.

- **AVG** (average scenario solution): It builds a solution assuming expected processing times $\bar{p}_i$ for each task $i \in I$.
- **MAX** (worst-case scenario solution): It builds a solution assuming maximum possible processing times $(1 + 1.5 \cdot \epsilon) \cdot \bar{p}_i$ for each task $i \in I$. It is a robust strategy that prepares for the worst-case scenario. It minimizes flight delays and overtime.
- **OB** (optimal buffer solution): An intermediate approach that sets task $i \in I$ processing time equal to $p_i = \gamma \cdot \bar{p}_i + (1 - \gamma) \cdot (1 + 1.5 \cdot \epsilon) \cdot \bar{p}_i$, where γ is a time buffer ranging between 0 and 1. For each instance, we choose the best performing γ value via an offline grid search. So, OB always outperforms both the AVG and MAX solutions.

6.1.2. Performance indicators

We measure the following Key Performance Indicators (KPIs) for each instance and solution (see Table 4).

Additionally, we compute the ratio of each solution algorithm's cost compared to the cost obtained by our best benchmark, OB (Solution Cost/OB Cost).

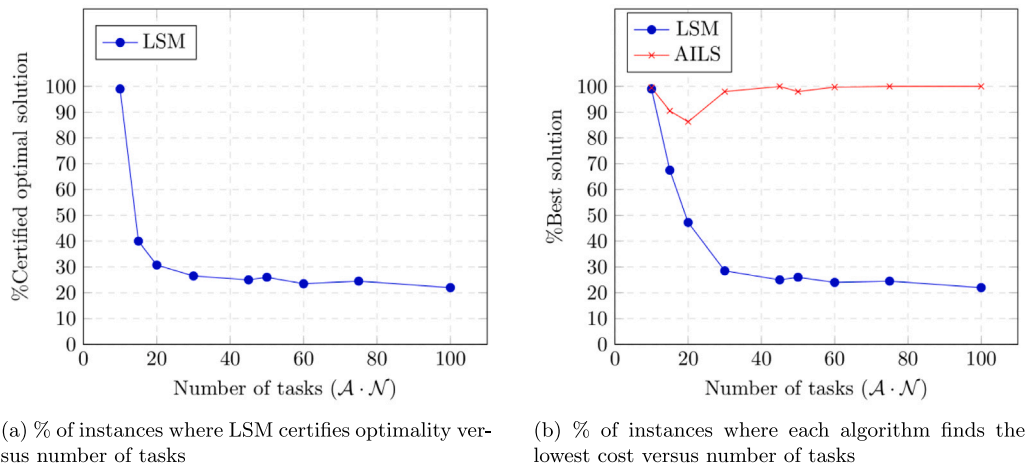


Fig. 6. Comparison of performance metrics versus number of tasks.

6.1.3. Experiments

We initially set a base experiment that studies the results of the 48 instance classes, aiming for two objectives. First, we aim to validate our heuristic solution compared to the Integer L-Shaped Method. Second, we aim to study the benefits of our heuristic solution compared to the proposed benchmarks.

Later, we built two additional experiments to study the impact of the system congestion and cost structure. These experiments are defined as follows:

- **Experiment 1: varying congestion levels.** We examine the impact of varying congestion levels in our solution's performance over instances with 15 aircraft and six technicians. Specifically, we build instances with different values $\mathcal{W} \in \{2, 4, 6\}$ and keep $\mathcal{A}$ and $\mathcal{T}$ fixed. For each $\mathcal{W}$ value, we set $\mathcal{N} = \mathcal{W}$ to avoid altering each task's average duration. These three possible settings with low ($\mathcal{W} = 2$), medium ($\mathcal{W} = 4$) and high ($\mathcal{W} = 6$) congestion levels are combined with all variability values $\mathcal{CV} \in \{0, 0.1, 0.25, 0.5\}$ obtaining 12 instance classes.
- **Experiment 2: varying cost structure.** Our base experiment sets cost weights to mainly minimize second-stage costs, but airlines have diverse objectives. In an intermediate instance class $\{\mathcal{A}, \mathcal{T}, \mathcal{W}, \mathcal{N}, \mathcal{CV}\} = \{15, 6, 4, 3, 0.25\}$, we set different cost weights for the first-stage and second-stage costs. This shows how our solution method adapts to different objective cost structures.

6.2. Results

This subsection presents the results of our base and extended experiments. Our solution was implemented in Python v3.8.8 programming language equipped with Gurobi v9.5.2 as the optimization solver. Solutions were computed on a server equipped with an Intel Xeon CPU E5-2630 v4 processor and 1 GB RAM of memory allocation. Also, we set a 1-hour time limit to solve each instance.

6.2.1. Base experiment results

Comparison of solution algorithms. First, we compare the solutions obtained using the exact L-shaped Method (LSM) with those obtained through our AILS heuristic, as presented in Sections 5.2 and 5.3, respectively. We discard a straightforward solution of the extended formulation, as it only solves small-sized deterministic instances to optimality. In Fig. 6(a), we present the percentage of instances in which the LSM certifies optimality for each total number of maintenance tasks ($\mathcal{A} \cdot \mathcal{N}$). For larger instances with more than 20 tasks, the LSM certifies optimality only in roughly 20% of the instances, mainly in those instances with deterministic task processing times (i.e., $\mathcal{CV} = 0$).

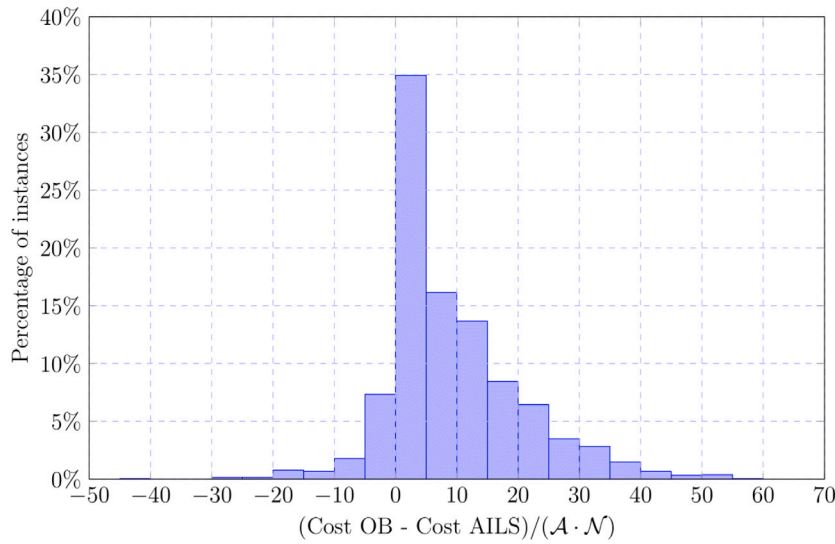
Fig. 6(b) presents the percentage of instances where each algorithm obtains the cheapest solution between the exact and heuristic algorithm for each total number of maintenance tasks. In each case, we limit computing time to one hour. Experimentally, the LSM beats AILS and certifies optimality in small-sized instances. However, AILS shows a competitive performance in these small instances. Moreover, AILS outperforms the LSM in larger-sized instances. Therefore, we focus our experimental results on the AILS algorithm.

Comparison to benchmark solutions. Now, we compare the solutions to our two-stage stochastic programming (TSSP) model (4) obtained via the AILS heuristic to each benchmark solution proposed. In Table 5, we present average results for each algorithm over all instances. On average, the TSSP based solution outperforms each of the three proposed benchmarks with an average total cost percentage savings (i.e., (benchmark cost - TSSP cost) / benchmark cost) of 74% compared to AVG, 34% compared to MAX, and 14% compared to OB. In parentheses next to each algorithm's average cost, we include the average cost ratio of each algorithm

Table 5

Average KPIs over all instances for each solution algorithm.

Solution Algorithm	AVG	MAX	OB	TSSP
Total cost (ratio to OB) [k\$]	13.34(3.25)	5.33(1.29)	4.11(1)	3.52(0.86)
Outsourcing cost [k\$]	0.57	3.44	1.86	1.29
Expected Flight delay cost [k\$]	12.68	1.87	2.22	2.20
Expected Overtime cost [k\$]	0.09	0.02	0.03	0.03
Outsourced task [%]	3.7%	22.1%	11.8%	7.9%
Outsourced task duration [min]	184.9	118.2	123.0	154.7
MOs used [#]	25.0	23.8	24.5	23.3
Pre-flight buffer [min]	4.9	26.3	18.3	20.7
Technician utilization [%]	61%	50%	56%	58%
Ground time utilization [%]	66%	56%	62%	63%

**Fig. 7.** Histogram of $(\text{Cost OB} - \text{Cost TSSP}) / (\mathcal{A} \cdot \mathcal{N})$ over all instances in the base experiment.

over OB.

A closer look at [Table 5](#) highlights the primary strategy of each benchmark solution. AVG yields, on average, the lowest outsourcing cost and the highest second-stage cost. This approach does not anticipate the second-stage cost variability. Thus, it maximizes the workload of in-house tasks and overload MOs by scheduling tasks too close to flight departures; this produces the smallest pre-flight buffer and the highest technician and ground time utilization. In contrast, MAX builds a conservative maintenance schedule with the highest percentage of outsourced tasks (22.1%) and the largest pre-flight buffer. As a result, it produces the lowest second-stage costs but the most expensive outsourcing cost. OB balances first and second-stage costs. Its best performing γ parameter value averages 0.73 over all instances; this is an intermediate value between the average ($\gamma = 1$) and the worst-case ($\gamma = 0$) solutions. Furthermore, the sampled coefficient of variation for the γ value chosen over all instances is 0.29 suggesting that γ is over-fitted to each instance. With respect to OB, TSSP further reduces outsourcing costs by increasing technician and aircraft ground time utilization. However, it keeps second-stage costs controlled (as opposed to AVG) and maintains relatively high pre-flight buffers.

See [Table 10](#) in [Appendix A.2](#) for detailed results by instance class.

To observe how TSSP outperforms OB in most instances, we present a histogram in [Fig. 7](#) showing the observed frequency of the expected cost difference over all instances. Compared to OB, TSSP produces solutions with cheaper or equal cost in 91% of the instances.

Task granularity. In our base experiment, we also study the impact of task granularity by classifying instances into three groups depending on how many tasks form the average workload per aircraft. Group 1 presents instances with a single task per aircraft ($\mathcal{N} = 1$) with an average duration of $\bar{p}_i = 210$ minutes per task. Group 2 divides this workload into three tasks per aircraft ($\mathcal{N} = 3$), each with an average duration of 70 min. Finally, group 3 presents five tasks per aircraft ($\mathcal{N} = 5$), each with an average duration of 42 min. In this way, we compare groups with an equal workload per aircraft but with different work fragmentation. Group 3 represents a setting more aligned with line maintenance practices, emphasizing the division of maintenance work into shorter, more manageable tasks rather than larger, consolidated checks, such as in group 1. In [Table 6](#), we present the average results obtained

Table 6

Average KPIs for the TSSP based solution over all instances in each task granularity group.

	1 task averaging 210 min each	3 tasks averaging 70 min each	5 tasks averaging 42 min each
Total cost (ratio to OB) [k\$]	8.90(0.99)	1.23(0.70)	0.44(0.29)
Outsourcing cost [k\$]	2.96	0.70	0.20
Expected Flight delay cost [k\$]	5.87	0.50	0.23
Expected Overtime cost [k\$]	0.07	0.03	0.01
Outsourced task [%]	19%	4%	1%
Outsourced task duration [min]	214.2	77.3	48.6
MOs used [#]	16	25	29
Pre-flight buffer [min]	23.5	19.1	19.3
Technician utilization [%]	52%	61%	61%
Ground time utilization [%]	60%	64%	64%

Table 7Average KPIs for the TSSP based solution over all instances for each $C\mathcal{V}$ case.

$C\mathcal{V}$	0	0.1	0.25	0.5
Total cost (ratio to OB) [k\$]	0.56(1)	0.92(0.74)	2.85(0.77)	9.78(0.90)
Outsourcing cost [k\$]	0.56	0.65	1.17	2.78
Expected Flight delay cost [k\$]	0	0.26	1.64	6.91
Expected Overtime cost [k\$]	0	0.01	0.04	0.09
Outsourced task [%]	4%	4%	7%	17%
Outsourced task duration [min]	216	204	156	116
MOs used [#]	25	24	23	22
Pre-flight buffer [min]	5	19	27	32
Technician utilization [%]	61%	61%	59%	52%
Ground time utilization [%]	67%	66%	64%	58%

by the TSSP based solution for each group.

In our experiment, a fragmented workload per aircraft significantly reduces maintenance costs due to a reduced percentage of outsourced tasks, explained by the increased number of MOs used, ground time utilization and percentage of in-house technician utilization. These findings highlight the benefits of fragmenting work, encouraging the adoption of line maintenance strategies.

Task processing time variability. Now, we study how our solution performs as a function of the coefficient of task processing time variability ($C\mathcal{V}$). Table 7 presents average results over each value of $C\mathcal{V}$. As expected, average costs increase as uncertainty grows. Also the OB solution yields costs identical to TSSP's for the deterministic instances. However, TSSP outperforms OB in uncertain instances ($C\mathcal{V} > 0$). Empirically, our results show that pre-flight buffers are longer as $C\mathcal{V}$ increases. This buffer works as a safety margin to accommodate unexpected delays in task processing times. Thus, TSSP adapts and extends safety margins as $C\mathcal{V}$ increases to shield against potential disruptions; this strategy, however, reduces effective technician and ground time utilization. We also observe that TSSP outsources tasks with relatively longer processing time when variability is low. As this variability grows, it outsources a higher percentage of tasks and also includes tasks with relatively shorter processing times. We observe similar behavior for OB, since its chosen γ value reduce as $C\mathcal{V}$ increases. Specifically, the γ values are set to 1, 0.70, 0.66, and 0.56 for $C\mathcal{V}$ values of 0, 0.1, 0.25, and 0.5, respectively.

6.2.2. Extended experiment 1: Technician congestion

Table 8 presents average results as a function of congestion level. As expected, costs increase as congestion rises. Relative to OB, TSSP achieves more benefits at medium congestion levels. In low congestion instances, TSSP and OB execute all tasks in-house. Consequently, outsourcing costs are zero, while flight delays and overtime costs are nearly zero. In this case, TSSP's marginal benefits over OB are small. In high congestion instances, TSSP outsources 15% of tasks mainly due to scarce and highly utilized technician resources. This leads to a significant cost increase. In this case, TSSP's absolute benefits over OB are higher.

6.2.3. Extended experiment 2: Cost structure

Now we solve all instances within class $\mathcal{A}, \mathcal{T}, \mathcal{W}, \mathcal{N}, C\mathcal{V} = \{15, 6, 4, 3, 0.25\}$ for each solution algorithm with varying cost structure determined by parameter ρ , which scales the second-stage costs. Specifically, we use the following objective function

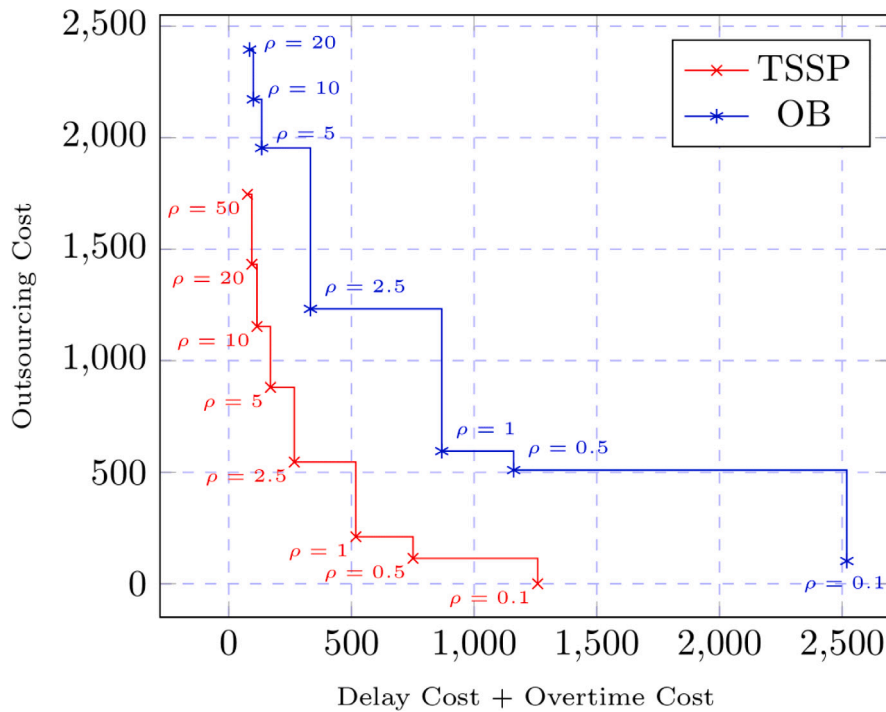
$$\min_{(x,y,z) \in \mathbb{X}} \sum_{i \in I} \sum_{o \in O_i} \beta_{i,o} \cdot (x_{i,o} - y_{i,o}) + \rho \cdot \mathbb{E}_{\omega}[Q(\mathbf{x}, \mathbf{z}, \omega)]. \quad (13)$$

Fig. 8 shows a scatter plot of the average outsourcing cost (first-stage) versus the average delay and overtime cost (second-stage) for multiple solutions for each ρ value and solution algorithm. This plot presents the trade-off between both objectives. For each solution,

Table 8

Average KPIs results for the TSSP based solution as a function congestion.

Congestion level	Low (30%–40%)	Medium (60%–75%)	High (90%–110%)
Total cost (ratio to OB) [k\$]	0.01(0.64)	0.67(0.43)	4.90(0.68)
Outsourcing cost [k\$]	0	0.36	4.02
Expected Flight delay cost [k\$]	0.01	0.28	0.81
Expected Overtime cost [k\$]	0	0.03	0.07
Outsourced task [%]	0%	2%	15%
Outsourced task duration [min]	–	66.5	61.9
MOs used [#]	21	25	26
Pre-flight buffer [min]	33.7	19.3	11.7
Technician utilization [%]	32%	61%	77%
Ground time utilization [%]	39%	63%	76%

**Fig. 8.** Scatter plot of outsourcing cost vs. flight delay and overtime costs.

a decrease in outsourcing costs leads to increased second-stage penalty costs. We also observe marginally decreasing second-stage cost savings for an increasing level of first-stage cost sacrifices. We observe how diverse solutions are depending on the objective. To the left, we identify solutions that minimize flight delays and overtime at the expense of an increase percentage of outsourced tasks. Conversely, towards the right, the focus shifts to minimizing tasks outsourced, even at the expense of larger values of flight delays and overtime. Both MAX and AVG benchmarks cannot adapt as a function of ρ ; the MAX benchmark yields an average outsourcing cost equal to \$4,033, with delay and overtime costs of \$52 and \$30, respectively. In contrast, the AVG benchmark executes all tasks with an expected second-stage cost equal to \$11,413. Conversely, the TSSP and OB solutions adapt as a function of ρ to align with the airline's objective. Compared to OB, TSSP yields a Pareto superior solution.

7. Conclusions

We study how to plan aircraft maintenance operations under uncertain task processing times over multiple bases and with a scarce technician workforce. To do it, we formulate this problem as a two-stage stochastic programming model. In the first stage, we assign each task to execute on each aircraft to a maintenance opportunity and schedule a sequence of in-house tasks for each technician. Then, we adjust each task start time and each flight departure time for each realized vector of task processing times in a second stage. In small-sized, this problem can be solved via a customized version of the L-shaped method for the LMSP. To

Table 9
Notation employed.

Sets	
U	Set of aircraft, indexed by $u \in U := \{1, \dots, U \}$.
F	Set of flights itineraries, indexed by $F \in F := \{F_1, \dots, F_{ U }\}$.
F_u	Flight itinerary of aircraft u , indexed by $f \in F_u := \{f_{u,1}, \dots, f_{u,q_u}\}$.
I	Set of tasks, indexed by $i \in I := \{1, \dots, I \}$.
I_0	Set of tasks extended, including a fictitious task 0.
A	Set of maintenance bases, indexed by $a \in A := \{1, \dots, A \}$.
A_i	Set of maintenance bases where task i can be executed.
O	Set of maintenance opportunities.
O_{start}	Set of maintenance opportunities that start before an aircraft's first flight.
O_{end}	Set of maintenance opportunities that start after an aircraft's last flight.
O_i	Set of maintenance opportunities where task i can be executed.
O_i^a	Set of maintenance opportunities at maintenance base a where task i can be executed.
Ω	Set of scenarios, indexed by $\omega \in \Omega = \{1, \dots, \Omega \}$.
Parameters	
$\beta_{i,o}$	Cost of outsourcing task i at MO o .
α_f	Delay cost per time unit for flight f .
μ	Fixed overtime cost incurred by a task ending after the operating horizon ends.
λ	Variable overtime cost per time unit incurred by a task ending after the operating horizon ends.
r_a	Number of technicians available at maintenance base a .
q_u	Number of flights to be performed by aircraft u .
h_f	Scheduled departure time of flight f .
τ_f	Sum of flight time and minimum required downtime for flight f .
$f^-(o)$	Flight that starts MO o with its landing.
$f^+(o)$	Flight that ends MO o with its departure.
T	Length of the operating horizon.
$\bar{p}_i$	Processing time of task i in the average scenario.
p_i^ω	Processing time of task i in scenario ω .
Decision variables	
$x_{i,o}$	Binary variable, 1 if task i is assigned to MO o , 0 otherwise.
$y_{i,o}$	Binary variable, 1 if task i is performed in-house at MO o , 0 otherwise.
$z_{i,j}^a$	Binary variable, 1 if task i precedes task j at maintenance base a , 0 otherwise.
$\bar{s}_i$	Start time of task i in the average scenario.
s_i^ω	Start time of task i in scenario ω .
$\bar{t}_f$	Departure time of flight f in scenario ω .
$t_f^{D,\omega}$	Delay of flight f in scenario ω .
u_i^ω	Overtime required to complete task i in scenario ω .
e_i^ω	Binary variable, 1 if task i begins within but concludes outside the operating horizon in scenario ω , otherwise 0.
l_i^ω	Binary variable, 1 if task i starts after the operating horizon in scenario ω , otherwise 0.

solve large-sized instances, we design an *ad-hoc* AILS heuristic that searches for good first-stage solutions and exactly evaluates the second-stage cost via dynamic programming. Our TSSP based solution is tested on a base experiment formed by 2400 simulated instances with varying levels of fleet size, number of technicians, workload per aircraft, number of maintenance tasks per aircraft, and task processing time variability.

In the base experiment, TSSP solution achieves average cost savings of 74% and 34% compared to deterministic solutions for the average and worst-case scenarios, respectively. Moreover, it yields a 14% average cost savings over a deterministic solution for the average scenario, but that adds an optimized time buffer to all task processing times.

We also learn several insights regarding aircraft maintenance planning. First, we assess the advantage of fragmenting the workload per aircraft into shorter tasks to obtain a better adjustment between maintenance resources and demand; this aligns with the main shift in the airline industry towards line maintenance. Second, we analyze how our solutions react as task processing time uncertainty grows. In uncertain environments, decision-makers should proactively increase ground time buffers to reduce potential flight departure delays.

In our experiments, the added value of relatively more complex stochastic optimization tools is relatively higher for intermediate workload congestion levels. Furthermore, our methodology can adapt solutions to the selected cost structure of a given company ranging from airlines minimizing outsourced tasks to others minimizing flight delays.

The presented model can be extended in several directions. We can include other sources of uncertainty, such as flight duration, technician absenteeism, and dynamic arrival of unexpected tasks (Lagos et al., 2020). Another extension is to enrich recourse actions once uncertainty is unveiled. For example, we may consider different task type and outsource non-critical tasks or reschedule the maintenance plan in the second stage decision. Finally, we may integrate maintenance scheduling with tail assignment decisions to enhance flexibility.

Table 10
Overall cost by instances classes.

$\mathcal{A}$	$\mathcal{T}$	$\mathcal{W}$	$\mathcal{N}$	$\mathcal{CV}$	AVG	MAX	OB	TSSP
10	4	4	1	0	2160	2160	2160	2160
				0.1	2864	3961	2719	2648
				0.25	9014	6439	4867	4865
				0.5	26 098	12 342	11 618	11 624
			3	0	6	6	6	6
				0.1	1065	1105	240	113
				0.25	6272	3237	1200	809
				0.5	26 996	5882	3218	2464
			5	0	3	3	3	3
				0.1	787	780	230	21
				0.25	4791	2599	810	241
				0.5	19 716	4944	2413	1305
15	6	4	1	0	1920	1920	1920	1920
				0.1	4004	5267	3193	3174
				0.25	12 974	10 691	9147	8967
				0.5	47 338	31 553	31 294	30 374
			3	0	0	0	0	0
				0.1	1652	560	130	23
				0.25	11 413	4115	1463	729
				0.5	45 749	8649	5486	4539
			5	0	0	0	0	0
				0.1	1452	486	452	1
				0.25	8621	3007	1529	72
				0.5	36 953	6792	3733	1424
20	4	2	1	0	720	720	720	720
				0.1	2083	3266	1331	1310
				0.25	11 487	5501	3679	3205
				0.5	38 040	8775	6581	6146
			3	0	6	6	6	6
				0.1	1415	960	467	1
				0.25	8663	2822	1204	256
				0.5	36 532	5065	2932	1475
			5	0	0	0	0	0
				0.1	1294	890	888	3
				0.25	7716	2409	1669	53
				0.5	29 919	4370	2621	567
20	8	4	1	0	1920	1920	1920	1920
				0.1	4304	6208	3555	3596
				0.25	17 188	15 201	13 211	12 897
				0.5	65 921	49 009	47 131	46 901
			3	0	0	0	0	0
				0.1	2395	1232	490	111
				0.25	15 339	5778	2879	1915
				0.5	62 010	11 870	8266	7234
			5	0	0	0	0	0
				0.1	2079	924	852	1
				0.25	11 289	3855	2890	150
				0.5	47 985	8906	5978	3260

CRedit authorship contribution statement

Matías Villafranca: Writing – original draft, Visualization, Validation, Methodology, Investigation, Formal analysis, Conceptualization. **Felipe Delgado:** Writing – original draft, Supervision, Methodology, Formal analysis, Conceptualization. **Mathias Klapp:** Writing – original draft, Supervision, Methodology, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix

A.1. Notation

See Table 9.

A.2. Results by instance classes

See Table 10.

References

- Ahmed, M.B., Ghroubi, W., Haouari, M., Sherali, H.D., 2017. A hybrid optimization-simulation approach for robust weekly aircraft routing and retiming. *Transp. Res. Part C: Emerg. Technol.* 84, 1–20. <http://dx.doi.org/10.1016/j.trc.2017.07.010>.
- Allamraju, H., 2021. pyflightdata. URL <https://pypi.org/project/pyflightdata/>.
- Ball, M., Barnhart, C., Dresner, M., Hansen, M., Neels, K., Odoni, A., Peterson, E., Sherry, L., Trani, A., Zou, B., 2010. Total delay impact study: a comprehensive assessment of the costs and impacts of flight delay in the United States. Technical report, University of California, Berkeley. Institute of Transportation Studies, URL <https://rosap.nrl.bts.gov/view/dot/6234>.
- Başdere, M., Bilge, Ü., 2014. Operational aircraft maintenance routing problem with remaining time consideration. *European J. Oper. Res.* 235 (1), 315–328. <http://dx.doi.org/10.1016/j.ejor.2013.10.066>.
- Beliën, J., Cardoen, B., Demeulemeester, E., 2012. Improving workforce scheduling of aircraft line maintenance at sabena technics. *Interfaces* 42 (4), 352–364. <http://dx.doi.org/10.1287/inte.1110.0585>.
- Benders, J., 1962. Partitioning procedures for solving mixed-variables programming problems. *Numer. Math.* 4 (1), 238–252. <http://dx.doi.org/10.1007/BF01386316>.
- Birge, J.R., Louveaux, F., 2011. *Introduction to Stochastic Programming*. Springer Science & Business Media, <http://dx.doi.org/10.1007/978-1-4614-0237-4>.
- Callewaert, P., Verhagen, W.J., Curran, R., 2018. Integrating maintenance work progress monitoring into aircraft maintenance planning decision support. *Transp. Res. Procedia* 29, 58–69. <http://dx.doi.org/10.1016/j.trpro.2018.02.006>.
- Caramia, M., Dell'Olmo, P., 2008. Coloring graphs by iterated local search traversing feasible and infeasible solutions. *Discrete Appl. Math.* 156 (2), 201–217. <http://dx.doi.org/10.1016/j.dam.2006.07.013>.
- Cook, A.J., Tanner, G., Anderson, S., 2004. Evaluating the True Cost to Airlines of One Minute of Airborne or Ground Delay. Technical report, Eurocontrol.
- Dayarian, I., Crainic, T.G., Gendreau, M., Rei, W., 2016. An adaptive large-neighborhood search heuristic for a multi-period vehicle routing problem. *Transp. Res. Part E: Logist. Transp. Rev.* 95, 95–123. <http://dx.doi.org/10.1016/j.jtre.2016.09.004>.
- De Bruecker, P., Beliën, J., Van den Bergh, J., Demeulemeester, E., 2018. A three-stage mixed integer programming approach for optimizing the skill mix and training schedules for aircraft maintenance. *European J. Oper. Res.* 267 (2), 439–452. <http://dx.doi.org/10.1016/j.ejor.2017.11.047>.
- De Bruecker, P., Van den Bergh, J., Beliën, J., Demeulemeester, E., 2015. A model enhancement heuristic for building robust aircraft maintenance personnel rosters with stochastic constraints. *European J. Oper. Res.* 246 (2), 661–673. <http://dx.doi.org/10.1016/j.ejor.2015.05.008>.
- Eurocontrol, 2023. Airline-reported all-cause delay for January 2017–December 2022 (All Column Values). DNM CODA Interactive Dashboard. Technical report, URL <https://ext.eurocontrol.int>.
- FAA, 2020. Cost of Delay Estimates 2019. Technical report, URL https://www.faa.gov/sites/faa.gov/files/data_research/aviation_data_statistics/cost_delay_estimates.pdf.
- Gendreau, M., Potvin, J.-Y., et al., 2010. *Handbook of metaheuristics*, vol. 2, Springer, <http://dx.doi.org/10.1007/978-1-4419-1665-5>.
- Gopalan, R., Talluri, K.T., 1998. The aircraft maintenance routing problem. *Oper. Res.* 46 (2), 260–271. <http://dx.doi.org/10.1287/opre.46.2.260>.
- Groër, C., Golden, B., Wasil, E., 2010. A library of local search heuristics for the vehicle routing problem. *Math. Program. Comput.* 2, 79–101. <http://dx.doi.org/10.1007/s12532-010-0013-5>.
- Gu, H., Zhang, Y., Zinder, Y., 2022. An efficient optimisation procedure for the workforce scheduling and routing problem: Lagrangian relaxation and iterated local search. *Comput. Oper. Res.* 144, 105829. <http://dx.doi.org/10.1016/j.cor.2022.105829>.
- Gupta, P., Bazargan, M., McGrath, R., 2003. Simulation model for aircraft line maintenance planning. In: *Annual Reliability and Maintainability Symposium*, 2003. IEEE, pp. 387–391. <http://dx.doi.org/10.1109/RAMS.2003.1182020>.
- He, Y., Ma, H.-L., Park, W.-Y., Liu, S.Q., Chung, S.-H., 2023. Maximizing robustness of aircraft routing with heterogeneous maintenance tasks. *Transp. Res. Part E: Logist. Transp. Rev.* 177, 103237. <http://dx.doi.org/10.1016/j.jtre.2023.103237>.
- IATA, 2018. Safety Report 2017. Technical report, URL <https://aviation-safety.net/airlinesafety/industry/reports/IATA-safety-report-2017.pdf>.
- IATA, 2022a. From Aircraft Health Monitoring to Aircraft Health Management White Paper on AHM. Technical report, URL <https://www.iata.org/contentassets/fafa409c883d41198aeb87628c848851/ahm-wp-1sted-2022.pdf>.
- IATA, 2022b. Airline Maintenance Cost Executive Commentary: FY2020 Data. Technical report, URL <https://www.iata.org/contentassets/bf8ca67c8bcd4358b3d004b0d6d0916f/fy2020-mctg-report-public.pdf>.
- IATA, 2023. IATA Annual Safety Report - 2022. Technical report, URL <https://www.iata.org/en/publications/safety-report/references/>.
- Kinnison, H.A., Siddiqui, T., 2013. *Aviation Maintenance Management*. McGraw-Hill Education.
- Kleywegt, A.J., Shapiro, A., Homem-de Mello, T., 2002. The sample average approximation method for stochastic discrete optimization. *SIAM J. Optim.* 12 (2), 479–502. <http://dx.doi.org/10.1137/S1052623499363220>.
- Lagos, C., Delgado, F., Klapp, M.A., 2020. Dynamic optimization for airline maintenance operations. *Transp. Sci.* 54 (4), 998–1015. <http://dx.doi.org/10.1287/trsc.2020.0984>.
- Laporte, G., Louveaux, F.V., 1993. The integer L-shaped method for stochastic integer programs with complete recourse. *Oper. Res. Lett.* 13 (3), 133–142. [http://dx.doi.org/10.1016/0167-6377\(93\)90002-X](http://dx.doi.org/10.1016/0167-6377(93)90002-X).
- Lapp, M., Cohn, A., 2012. Modifying lines-of-flight in the planning process for improved maintenance robustness. *Comput. Oper. Res.* 39 (9), 2051–2062. <http://dx.doi.org/10.1016/j.cor.2011.08.024>.
- Liang, Z., Chaovalitwongse, W.A., Huang, H.C., Johnson, E.L., 2011. On a new rotation tour network model for aircraft maintenance routing problem. *Transp. Sci.* 45 (1), 109–120. <http://dx.doi.org/10.1287/trsc.1100.0338>.
- Liang, Z., Feng, Y., Zhang, X., Wu, T., Chaovalitwongse, W.A., 2015. Robust weekly aircraft maintenance routing problem and the extension to the tail assignment problem. *Transp. Res. Part B: Methodol.* 78, 238–259. <http://dx.doi.org/10.1016/j.trb.2015.03.013>.
- Ma, H.-L., Sun, Y., Chung, S.-H., Chan, H.K., 2022. Tackling uncertainties in aircraft maintenance routing: A review of emerging technologies. *Transp. Res. Part E: Logist. Transp. Rev.* 164, 102805. <http://dx.doi.org/10.1016/j.jtre.2022.102805>.

- Maier, S.J., Desaulniers, G., Soumis, F., 2014. Recoverable robust single day aircraft maintenance routing problem. *Comput. Oper. Res.* 51, 130–145. <http://dx.doi.org/10.1016/j.cor.2014.03.007>.
- Maier, S.J., Desaulniers, G., Soumis, F., 2018. The daily tail assignment problem under operational uncertainty using look-ahead maintenance constraints. *European J. Oper. Res.* 264 (2), 534–547. <http://dx.doi.org/10.1016/j.ejor.2017.06.041>.
- McNabb, M.E., Weir, J.D., Hill, R.R., Hall, S.N., 2015. Testing local search move operators on the vehicle routing problem with split deliveries and time windows. *Comput. Oper. Res.* 56, 93–109. <http://dx.doi.org/10.1016/j.cor.2014.11.007>.
- Papakostas, N., Papachatzakis, P., Xanthakis, V., Mourtzis, D., Chrysosolouris, G., 2010. An approach to operational aircraft maintenance planning. *Decis. Support Syst.* 48 (4), 604–612. <http://dx.doi.org/10.1016/j.dss.2009.11.010>.
- Pflug, G.C., Pichler, A., 2014. *Multistage Stochastic Optimization*, vol. 1104, Springer, <http://dx.doi.org/10.1007/978-3-319-08843-3>.
- Pisinger, D., Ropke, S., 2007. A general heuristic for vehicle routing problems. *Comput. Oper. Res.* 34 (8), 2403–2435. <http://dx.doi.org/10.1016/j.cor.2005.09.012>.
- Prentice, B., DiNota, A., Constanza, D., Reagan, I., Franzoni, C., Stelle, M., 2022. Global Fleet & MRO Market Forecast Commentary 2022–2032. Technical report, Oliver Wyman, URL https://www.oliverwyman.com/content/dam/oliver-wyman/v2/publications/2022/feb/MRO-2022-Master-file_v5.pdf.
- Qantas, 2016. The A, C and D of Aircraft Maintenance. Technical report, URL <https://www.qantasnewsroom.com.au/roo-tales/the-a-c-and-d-of-aircraft-maintenance/>.
- Ruther, S., Boland, N., Engineer, F.G., Evans, I., 2017. Integrated aircraft routing, crew pairing, and tail assignment: Branch-and-price with many pricing problems. *Transp. Sci.* 51 (1), 177–195. <http://dx.doi.org/10.1287/trsc.2015.0664>.
- Safaei, N., Banjevic, D., Jardine, A.K., 2011. Workforce-constrained maintenance scheduling for military aircraft fleet: a case study. *Ann. Oper. Res.* 186, 295–316. <http://dx.doi.org/10.1007/s10479-011-0885-4>.
- Safaei, N., Jardine, A.K., 2018. Aircraft routing with generalized maintenance constraints. *Omega* 80, 111–122. <http://dx.doi.org/10.1016/j.omega.2017.08.013>.
- Saltzman, R.M., Stern, H.I., 2022. The multi-day aircraft maintenance routing problem. *J. Air Transp. Manag.* 102, 102224. <http://dx.doi.org/10.1016/j.jairtraman.2022.102224>.
- Shahmoradi-Moghadam, H., Safaei, N., Sadjadi, S.J., 2021. Robust maintenance scheduling of aircraft fleet: A hybrid simulation-optimization approach. *IEEE Access* 9, 17854–17865. <http://dx.doi.org/10.1109/ACCESS.2021.3053714>.
- Shapiro, A., 2003. Monte Carlo sampling methods. *Handbooks Oper. Res. Management Sci.* 10, 353–425. [http://dx.doi.org/10.1016/S0927-0507\(03\)10006-0](http://dx.doi.org/10.1016/S0927-0507(03)10006-0).
- Shapiro, A., Dentcheva, D., Ruszczyński, A., 2014. *Lectures on Stochastic Programming: Modeling and Theory*, Second Edition. Society for Industrial and Applied Mathematics, Philadelphia, PA, <http://dx.doi.org/10.1137/1.9781611973433>, URL <https://epubs.siam.org/doi/abs/10.1137/1.9781611973433>.
- Shaukat, S., Katscher, M., Wu, C.-L., Delgado, F., Larrain, H., 2020. Aircraft line maintenance scheduling and optimisation. *J. Air Transp. Manag.* 89, 101914. <http://dx.doi.org/10.1016/j.jairtraman.2020.101914>.
- Vaessens, R.J.M., Aarts, E.H.L., Lenstra, J.K., 1996. Job shop scheduling by local search. *Inform. J. Comput.* 8 (3), 302–317. <http://dx.doi.org/10.1287/ijoc.8.3.302>.
- Van den Bergh, J., De Bruecker, P., Beliën, J., Peeters, J., 2013. Aircraft maintenance operations: state of the art. HUB Research Paper 2013/09, URL <https://lirias.kuleuven.be/1677212?limo=0>.
- Van Slyke, R.M., 1963. Monte Carlo methods and the PERT problem. *Oper. Res.* 11 (5), 839–860. <http://dx.doi.org/10.1287/opre.11.5.839>.